Advanced Data Structures
CS4580

Lecture 01

What The Course Is About

- Study data structures for:
  - External sorting
  - Single and double ended priority queues
  - Dictionaries
  - Multidimensional search
  - Computational geometry
  - Image processing
  - Packet routing and classification

What The Course Is About

- Concerned with:
  - Worst-case complexity
  - Average complexity
  - Amortized complexity

Prerequisites

- ✓ C++ / Java / Python
- ✓ Asymptotic Complexity
  - Big Oh, Theta, and Omega notations
- ✓ Undergraduate data structures
  - Stacks and Queues
  - Linked lists
  - Trees
  - Graphs

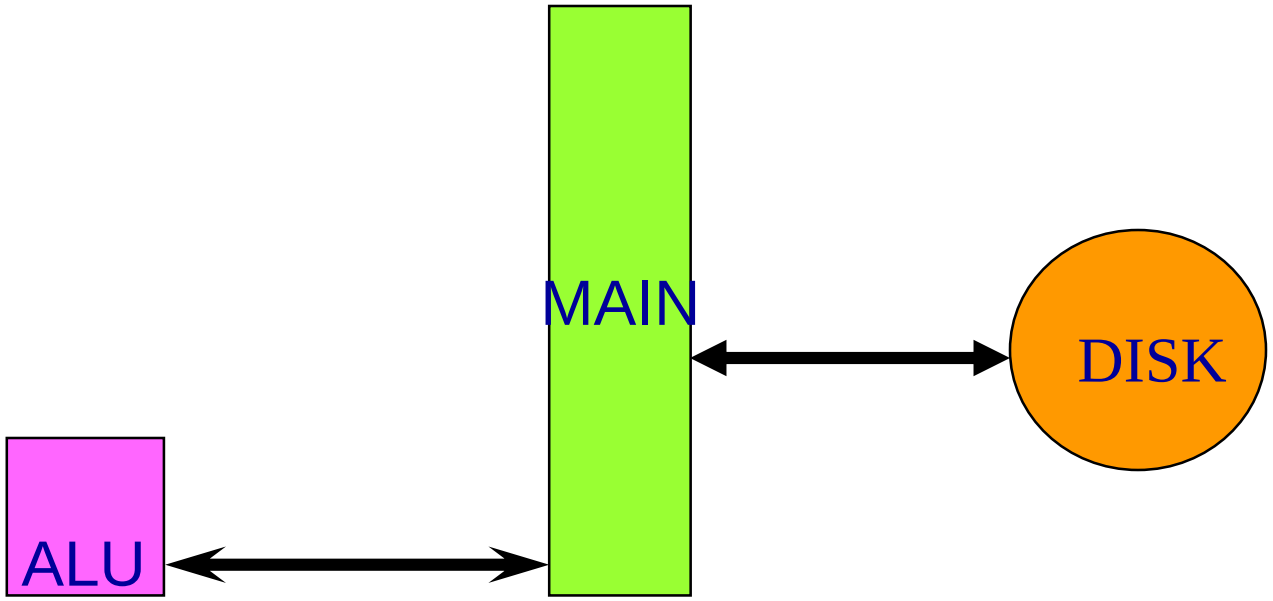
External Sorting

- Sort n records/elements that reside on a disk.
- Space needed by the n records is very large.
  - n is very large, and each record may be large or small.
  - n is small, but each record is very large.
- So, not feasible to input the n records, sort, and output in sorted order.

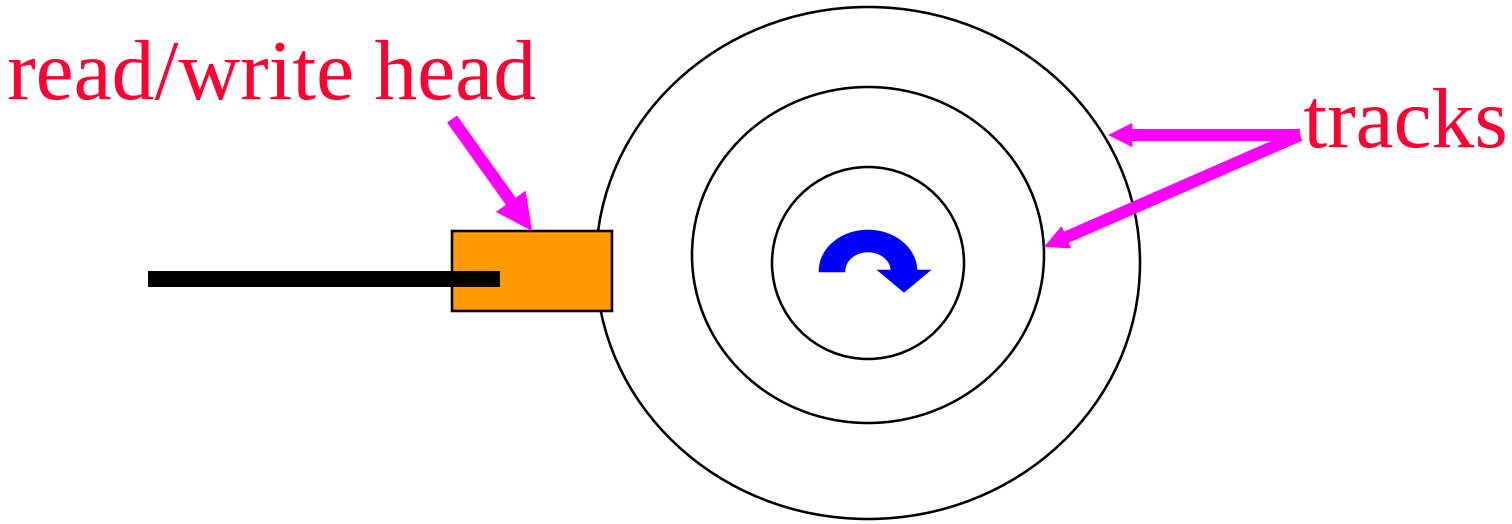
New Data Structures/Concepts

- Tournament trees.
- Huffman trees.
- Double-ended priority queues.
- Buffering.

External Sort Computer Model

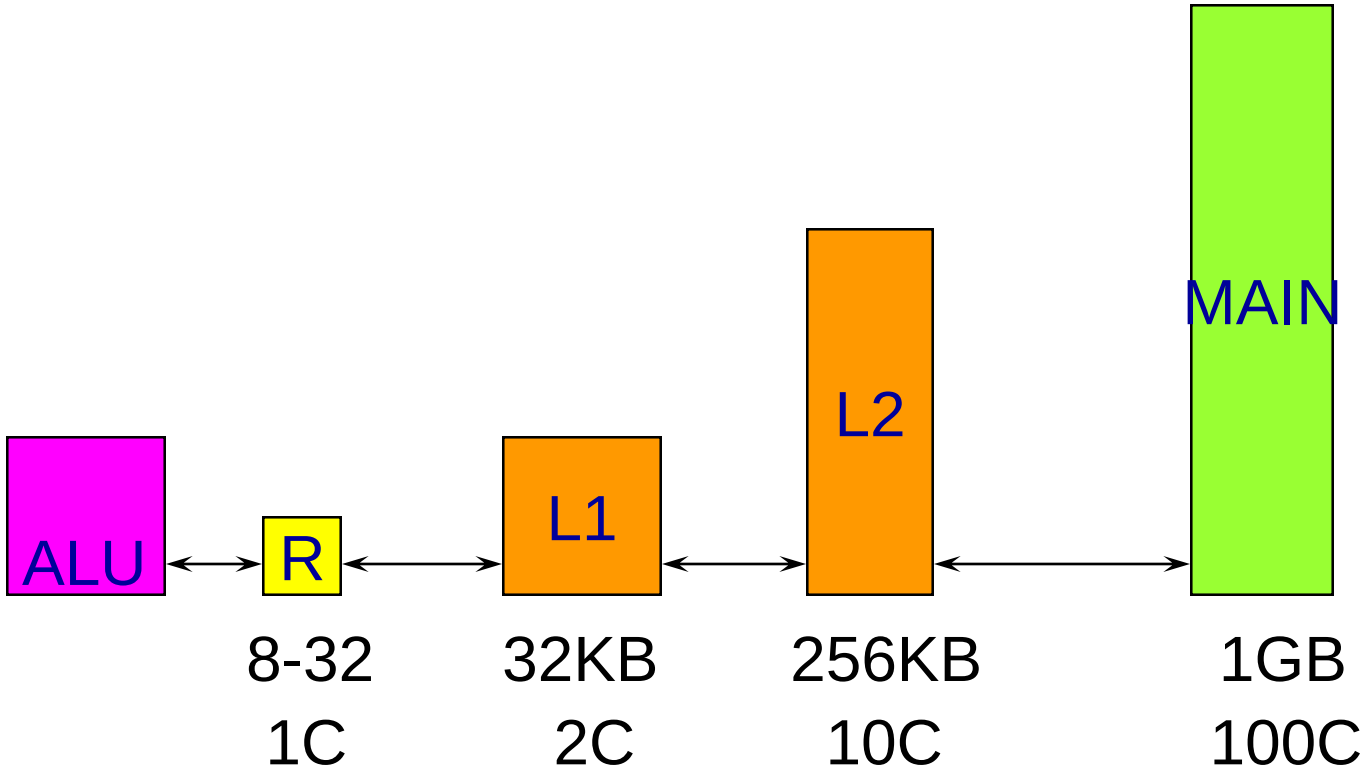


Disk Characteristics



- Seek time
  - Approx. 100,000 arithmetics
- Latency time
  - Approx. 25,000 arithmetics
- Transfer time
- Data access by block

Memory Model



External Sort Methods

- Base the external sort method on a fast internal sort method.
- Average run time
  - Quick sort
- Worst-case run time
  - Merge sort

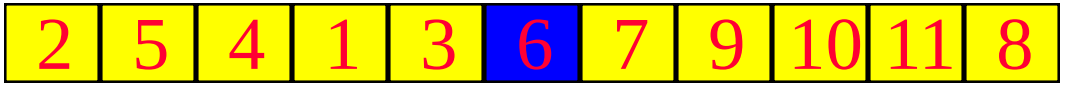
Internal Quick Sort

- To sort a large instance, select a pivot element from out of the n elements.
- Partition the n elements into 3 groups left, middle and right.
- The middle group contains only the pivot element.
- All elements in the left group are <= pivot.
- All elements in the right group are >= pivot.
- Sort left and right groups recursively.
- Answer is sorted left group, followed by middle group followed by sorted right group.

Internal Quick Sort

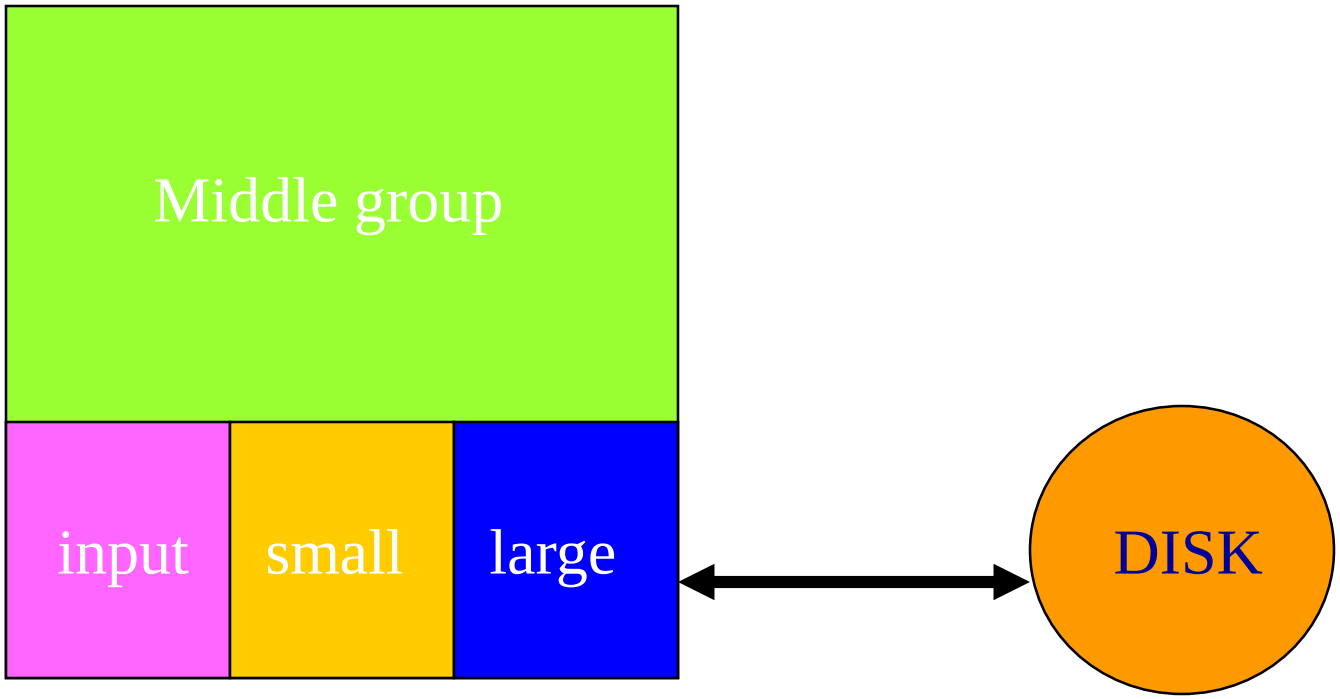


Use 6 as the pivot.



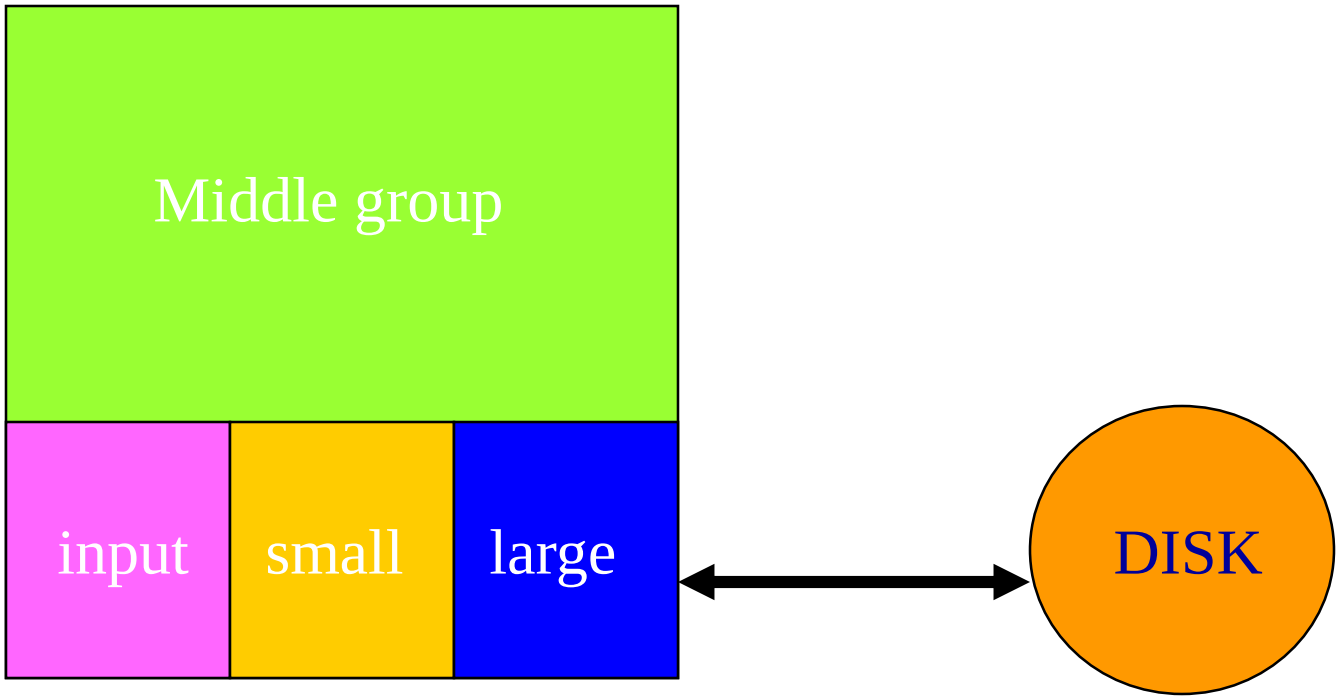
Sort left and right groups recursively.

Quick Sort – External Adaptation



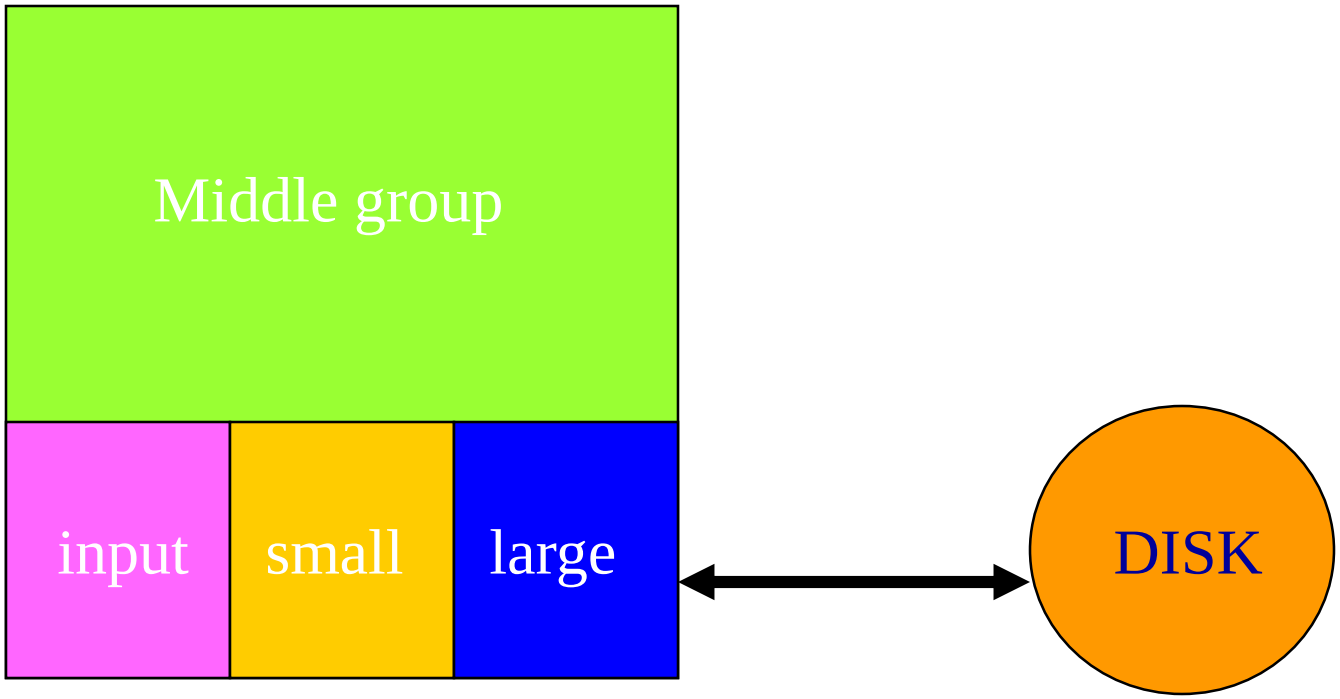
- 3 input/output buffers
  - input, small, large
- rest is used for middle group

Quick Sort – External Adaptation



- fill middle group from disk
- if next record <= $middle_{min}$ send to small
- if next record >= $middle_{max}$ send to large
- else remove $middle_{min}$ or $middle_{max}$ from middle and add new record to middle group

Quick Sort – External Adaptation



- Fill input buffer when it gets empty.
- Write small/large buffer when full.
- Write middle group in sorted order when done.
- Double-ended priority queue.
- Use additional buffers to reduce I/O wait time.

External Sorting

- Adapt fastest internal-sort methods.

✓Quick sort …best average run time.

- Merge sort … best worst-case run time.

Internal Merge Sort Review

- Phase 1
  - Create initial sorted segments
    - Natural segments
    - Insertion sort
- Phase 2
  - Merge pairs of sorted segments, in merge passes, until only 1 segment remains.

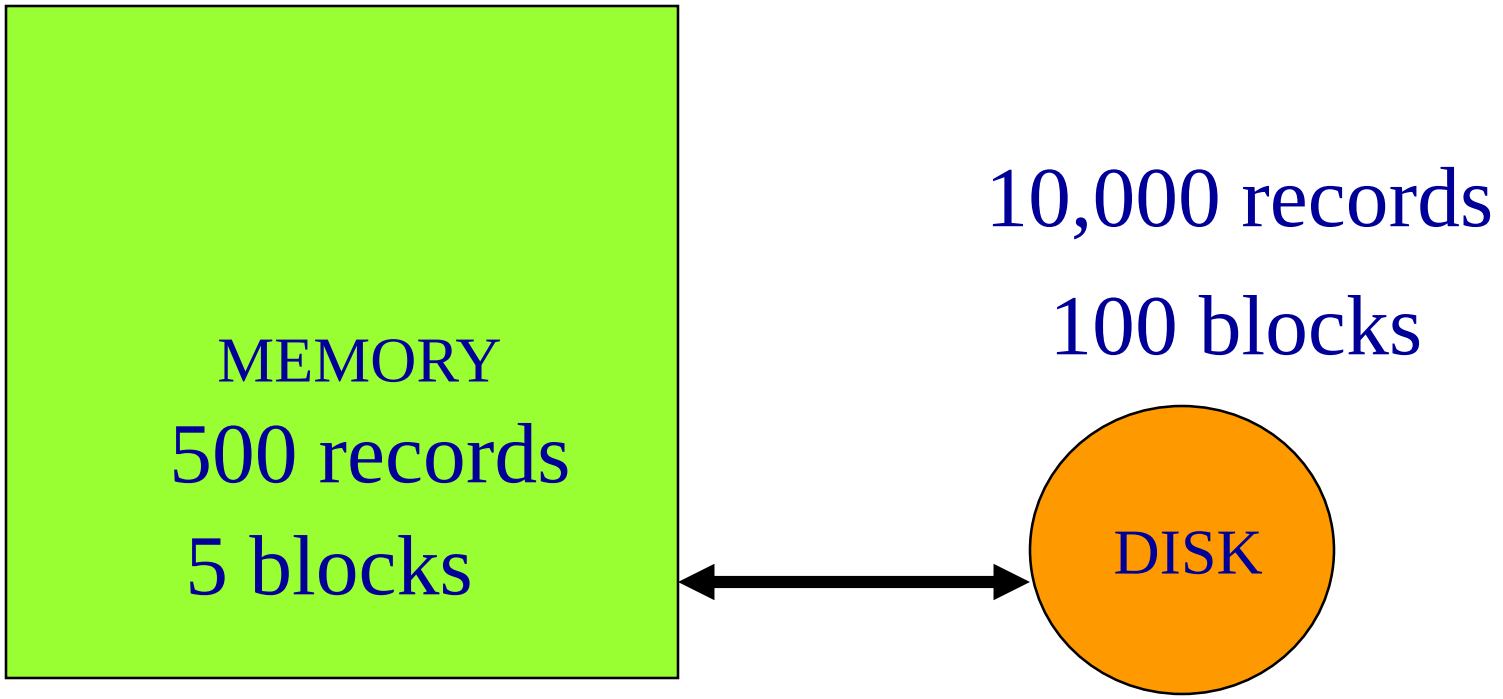
External Merge Sort

- Sort 10,000 records.
- Enough memory for 500 records.
- Block size is 100 records.
- t_{IO} = time to input/output 1 block
  (includes seek, latency, and transmission times)
- t_{IS} = time to internally sort 1 memory load
- t_{IM} = time to internally merge 1 block load

External Merge Sort

- Two phases.
  - Run generation.
    - A run is a sorted sequence of records.
  - Run merging.

Run Generation



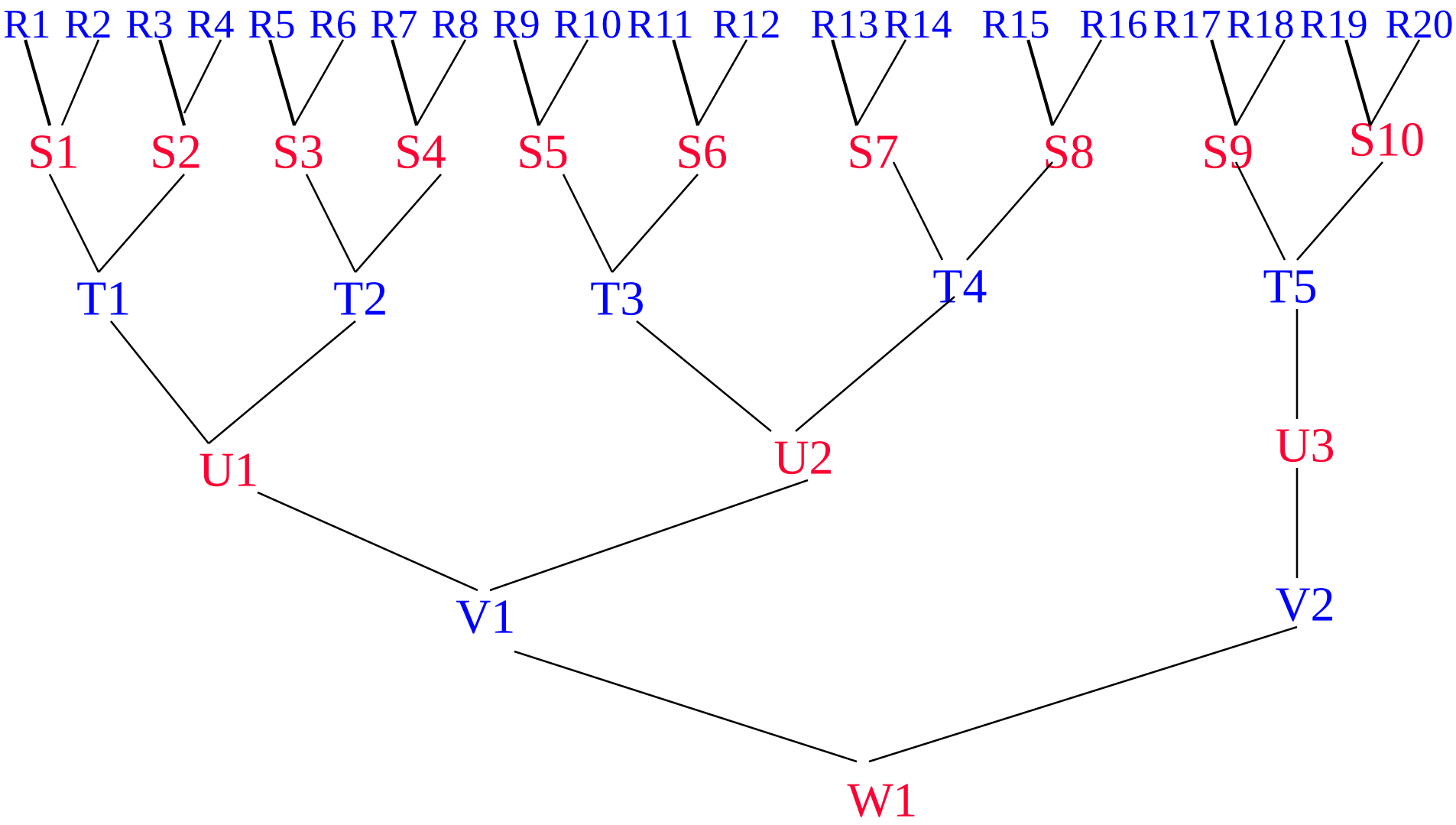
- Input 5 blocks.
- Sort.
- Output as a run.
- Do 20 times.

- $5t_{IO}$
- t_{IS}
- $5t_{IO}$
- $200t_{IO} + 20t_{IS}$

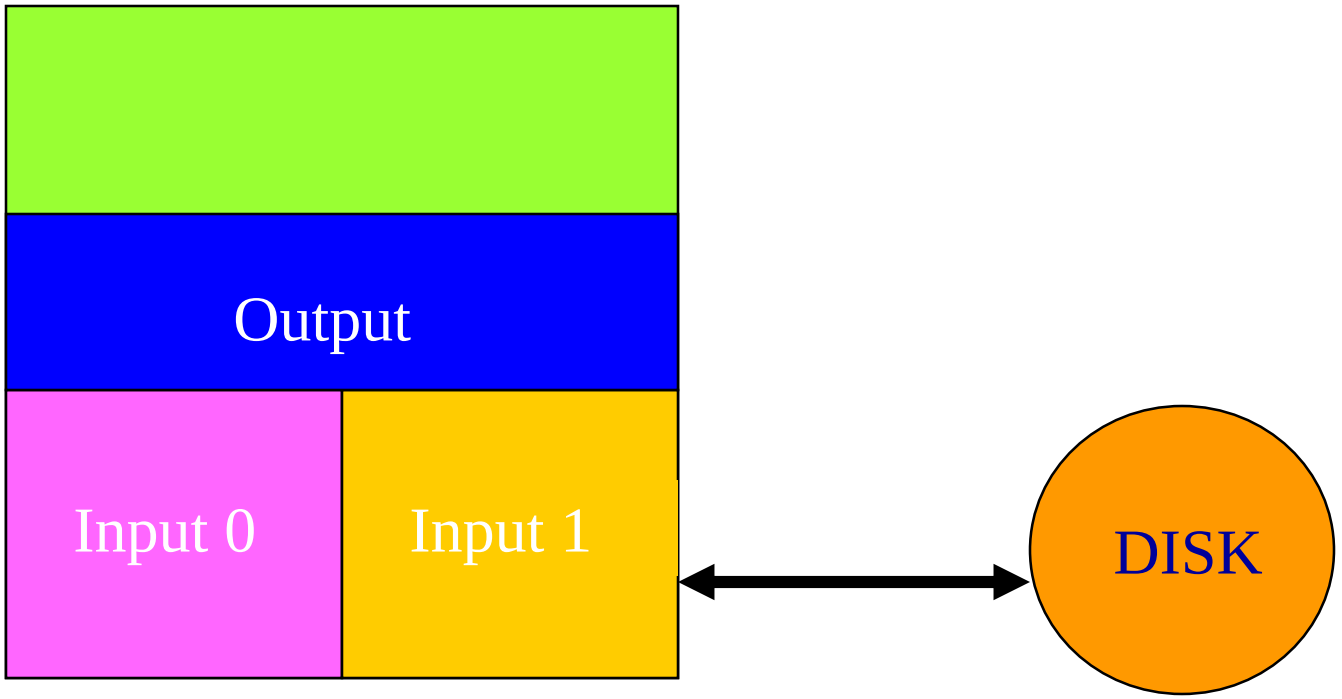
Run Merging

- Merge Pass.
  - Pairwise merge the 20 runs into 10.
  - In a merge pass all runs (except possibly one) are pairwise merged.
- Perform 4 more merge passes, reducing the number of runs to 1.

Merge 20 Runs



Merge R1 and R2



- Fill I0 (Input 0) from R1 and I1 from R2.
- Merge from I0 and I1 to output buffer.
- Write whenever output buffer full.
- Read whenever input buffer empty.

Time To Merge R1 and R2

- Each is 5 blocks long.
- Input time = $10t_{IO}$.
- Write/output time = $10t_{IO}$.
- Merge time = $10t_{IM}$.
- Total time = $20t_{IO} + 10t_{IM}$.

Time For Pass 1 (R → S)

- Time to merge one pair of runs
  $= 20t_{IO} + 10t_{IM}$.
- Time to merge all 10 pairs of runs
  $= 200t_{IO} + 100t_{IM}$.

Time To Merge S1 and S2

- Each is 10 blocks long.
- Input time = $20t_{IO}$.
- Write/output time = $20t_{IO}$.
- Merge time = $20t_{IM}$.
- Total time = $40t_{IO} + 20t_{IM}$.

Time For Pass 2 (S → T)

- Time to merge one pair of runs
  $= 40t_{IO} + 20t_{IM}$.
- Time to merge all 5 pairs of runs
  $= 200t_{IO} + 100t_{IM}$.

Time For One Merge Pass

- Time to input all blocks = $100t_{IO}$.
- Time to output all blocks = $100t_{IO}$.
- Time to merge all blocks = $100t_{IM}$.
- Total time for a merge pass = $200t_{IO} + 100t_{IM}$.

Total Run-Merging Time

- (time for one merge pass) * (number of passes)

  = (time for one merge pass)

  * ceil($\log_2$(number of initial runs))

  = $(200t_{IO} + 100t_{IM})$ * ceil($\log_2(20)$)

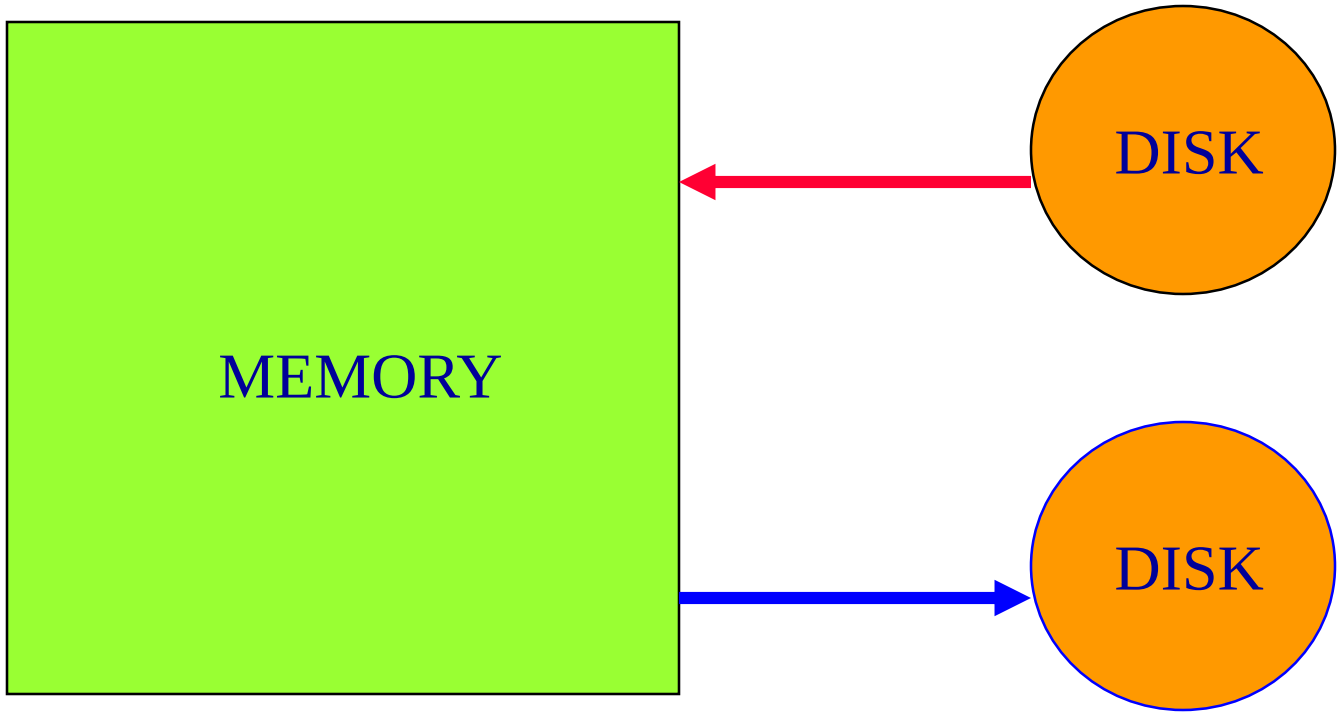
  = $(200t_{IO} + 100t_{IM})$ * 5

Factors In Overall Run Time

- Run generation. $200t_{IO} + 20t_{IS}$
  - Internal sort time.
  - Input and output time.
- Run merging. $(200t_{IO} + 100t_{IM}) * \text{ceil}(\log_2(20))$
  - Internal merge time.
  - Input and output time.
  - Number of initial runs.
  - Merge order (number of merge passes is determined by number of runs and merge order)

Improve Run Generation

- Overlap input, output, and internal sorting.

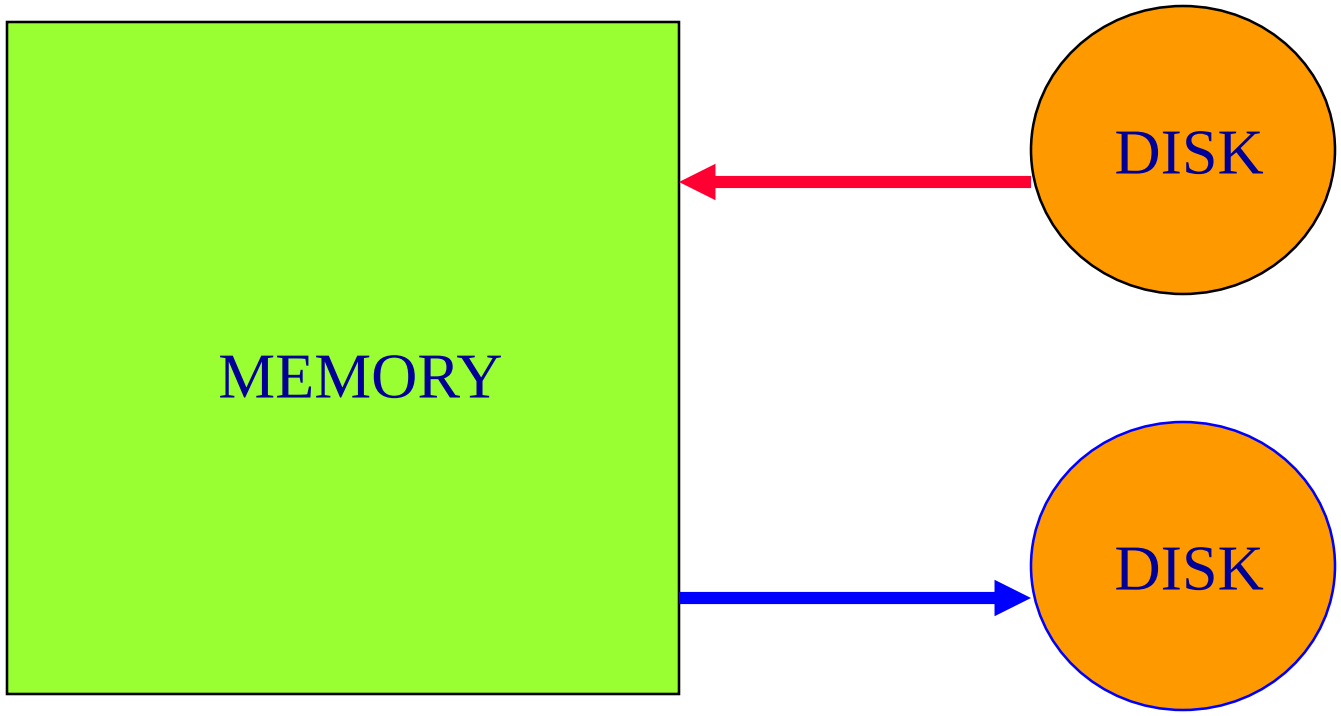


Improve Run Generation

- Generate runs whose length (on average) exceeds memory size.
- Equivalent to reducing number of runs generated.

Improve Run Merging

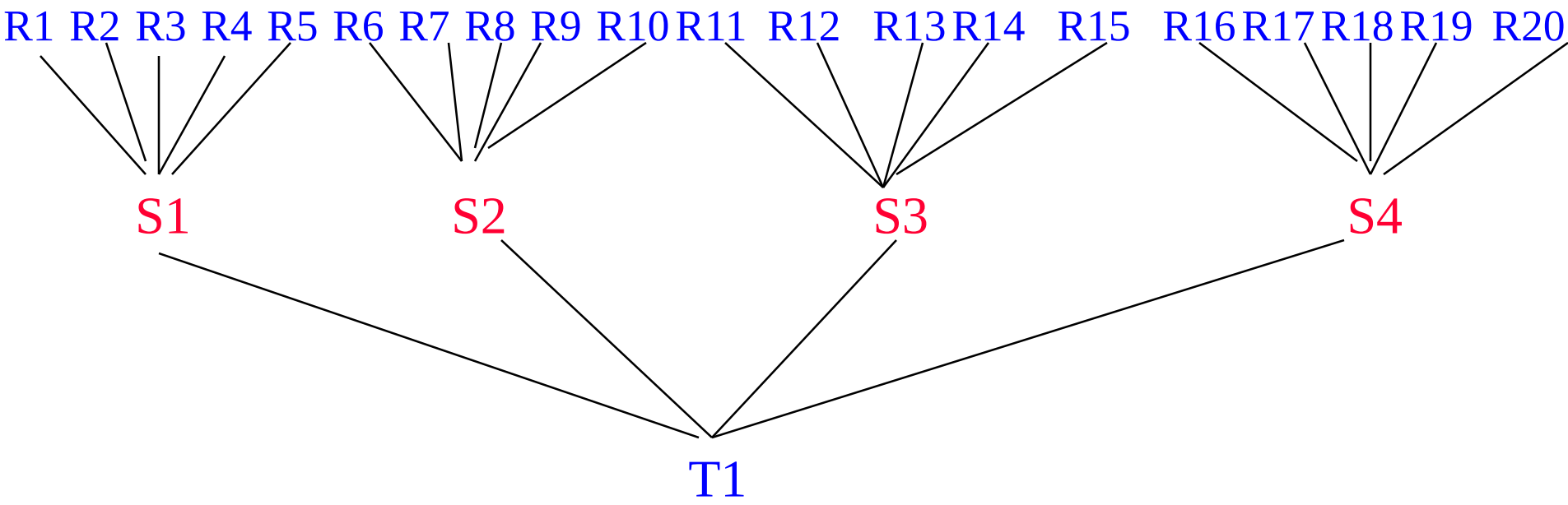
- Overlap input, output, and internal merging.



Improve Run Merging

- Reduce number of merge passes.
  - Use higher-order merge.
  - Number of passes
    = ceil($\log_k$(number of initial runs))
    where k is the merge order.

Merge 20 Runs Using 5-Way Merging

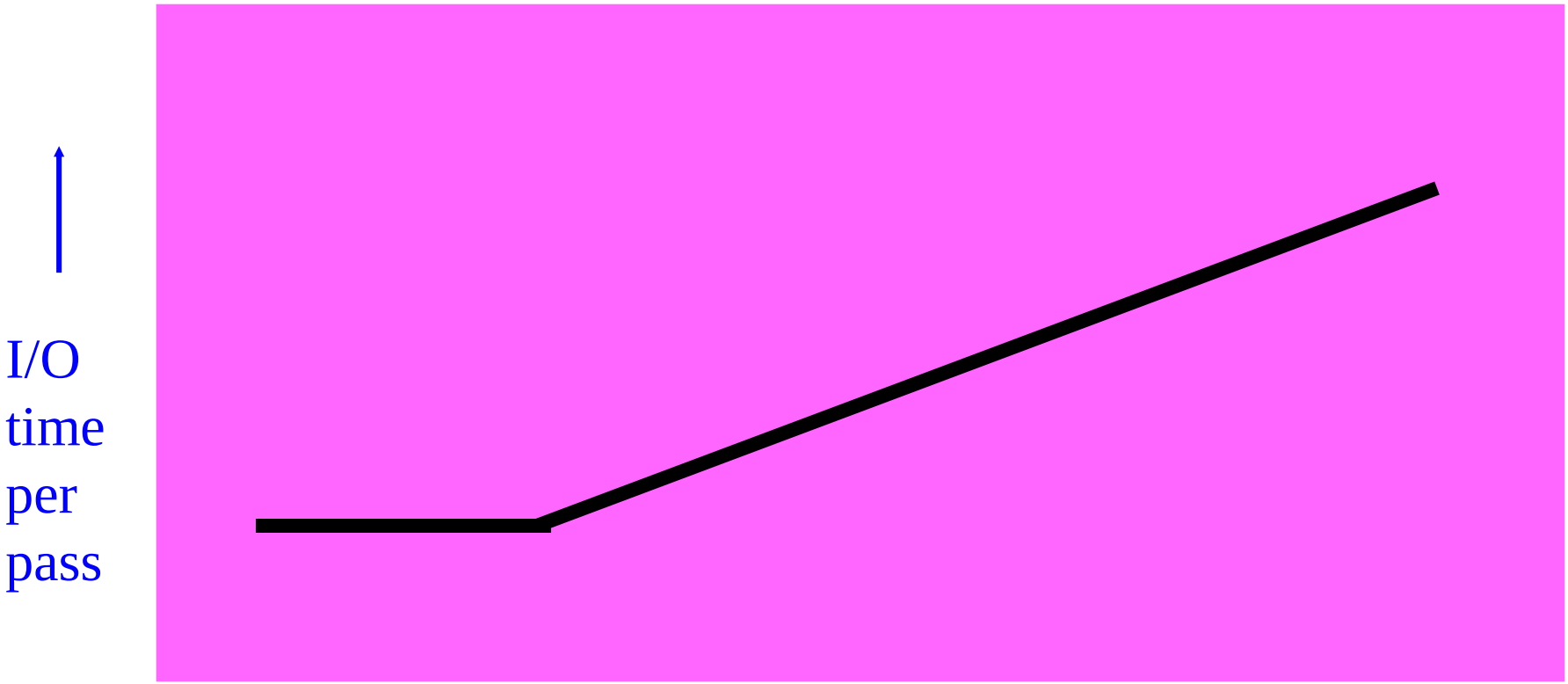


Number of passes = 2

I/O Time Per Merge Pass

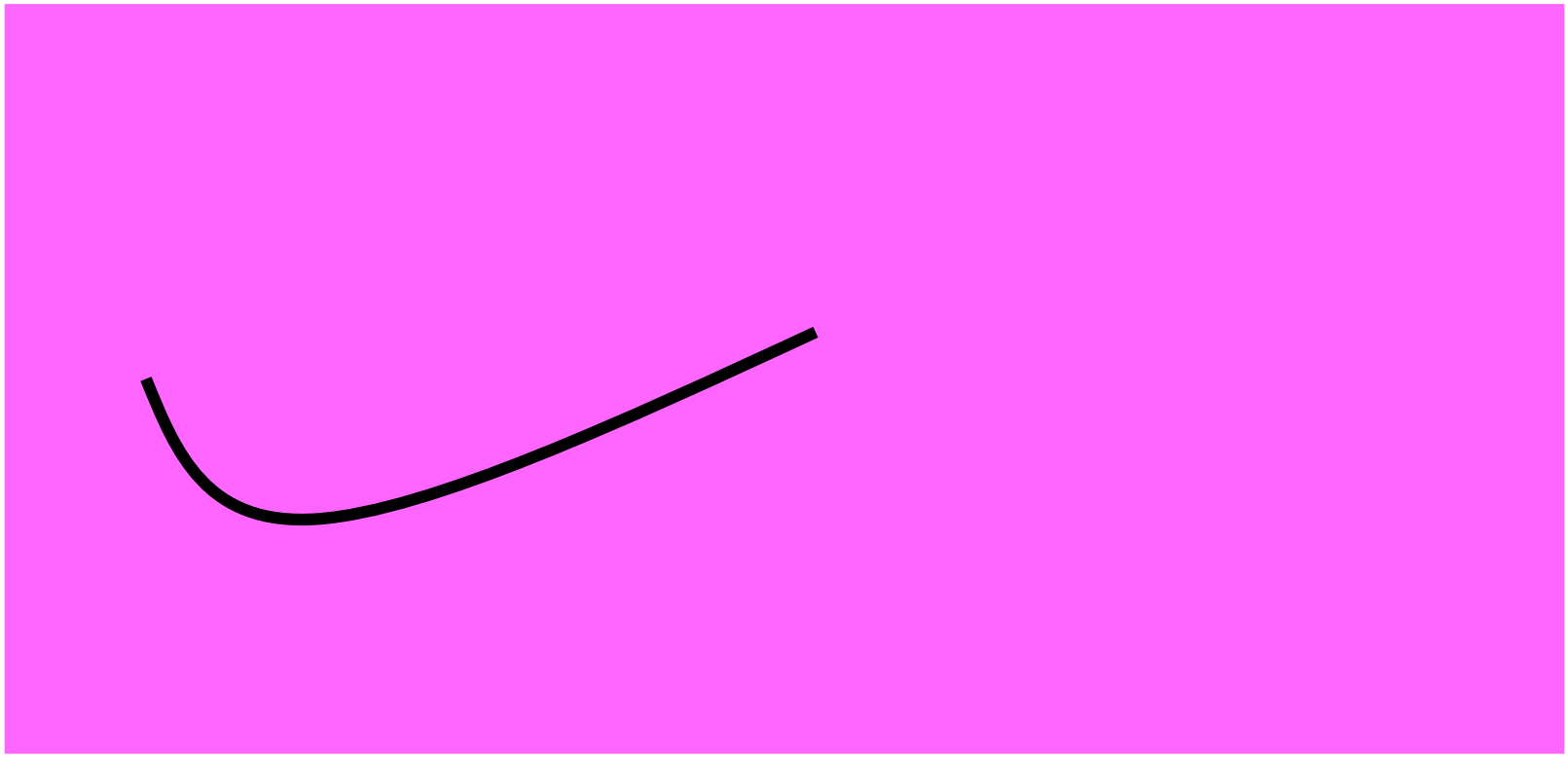
- Number of input buffers needed is linear in merge order k.
- Since memory size is fixed, block size decreases as k increases (after a certain k).
- So, number of blocks increases.
- So, number of seek and latency delays per pass increases.

I/O Time Per Merge Pass

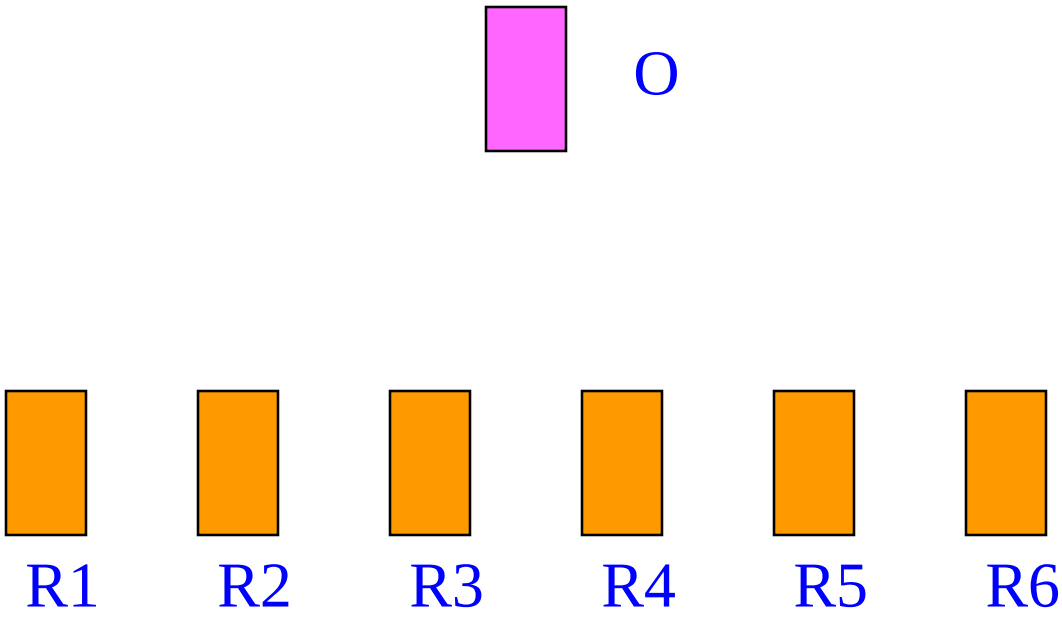


Total I/O Time To Merge Runs

- (I/O time for one merge pass)
  * ceil($\log_k$(number of initial runs))

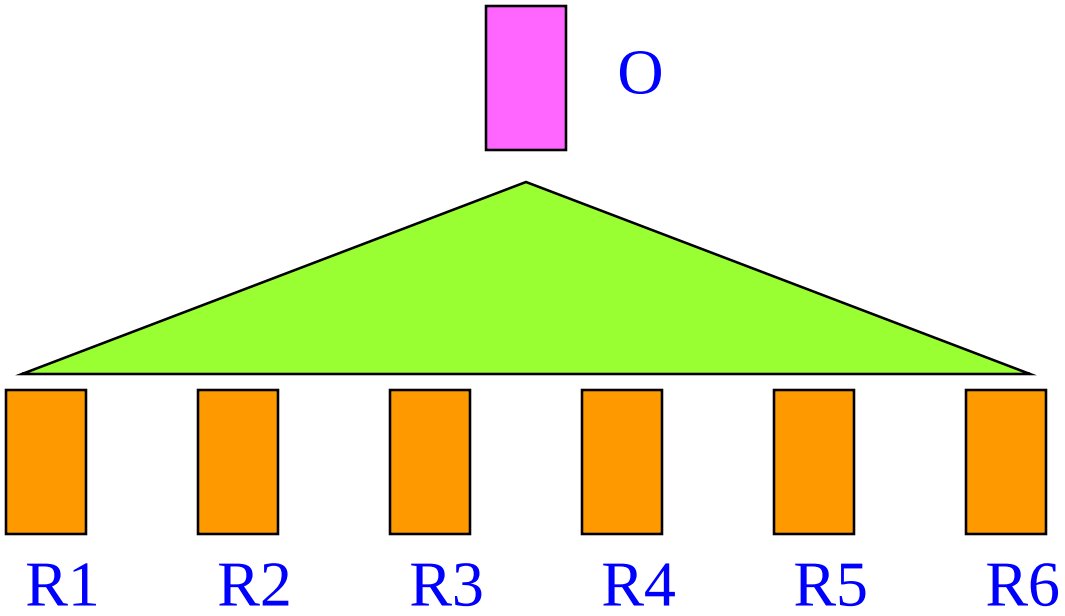


Internal Merge Time



- Naïve way => $k - 1$ compares to determine next record to move to the output buffer.
- Time to merge n records is $c(k - 1)n$, where c is a constant.
- Merge time per pass is $c(k - 1)n$.
- Total merge time is $c(k - 1)n\log_k r \sim cn(k/\log_2 k)\log_2 r$.

Merge Time Using A Tournament Tree



- Time to merge n records is $dn\log_2 k$, where d is a constant.
- Merge time per pass is $dn\log_2 k$.
- Total merge time is $(dn\log_2 k)\ \log_k r = dn\log_2 r$.

Tournament Trees

Winner trees.

Loser Trees.

Winner Tree – Definition

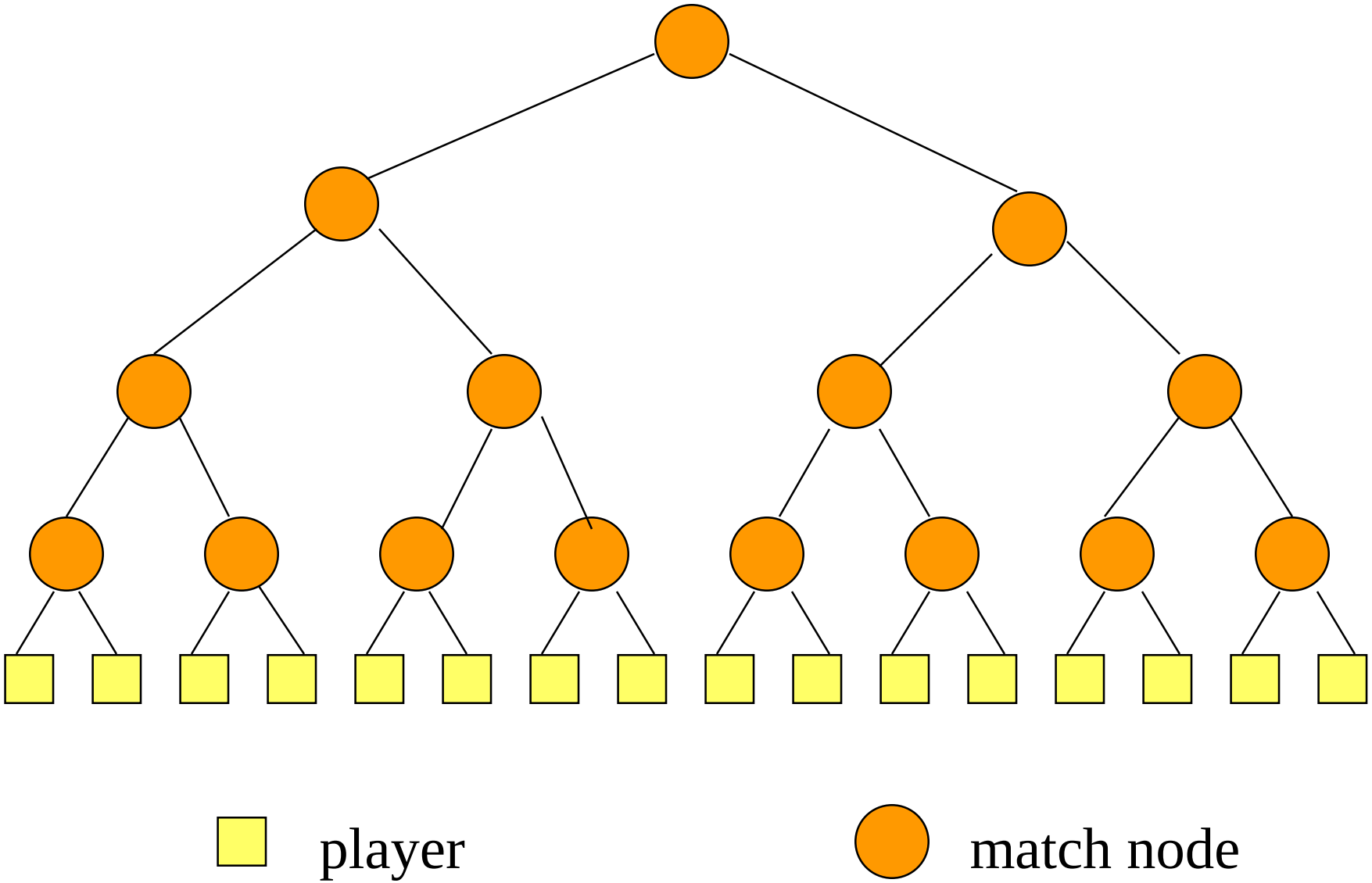
Complete binary tree with n external nodes and n – 1 internal nodes.

External nodes represent tournament players.

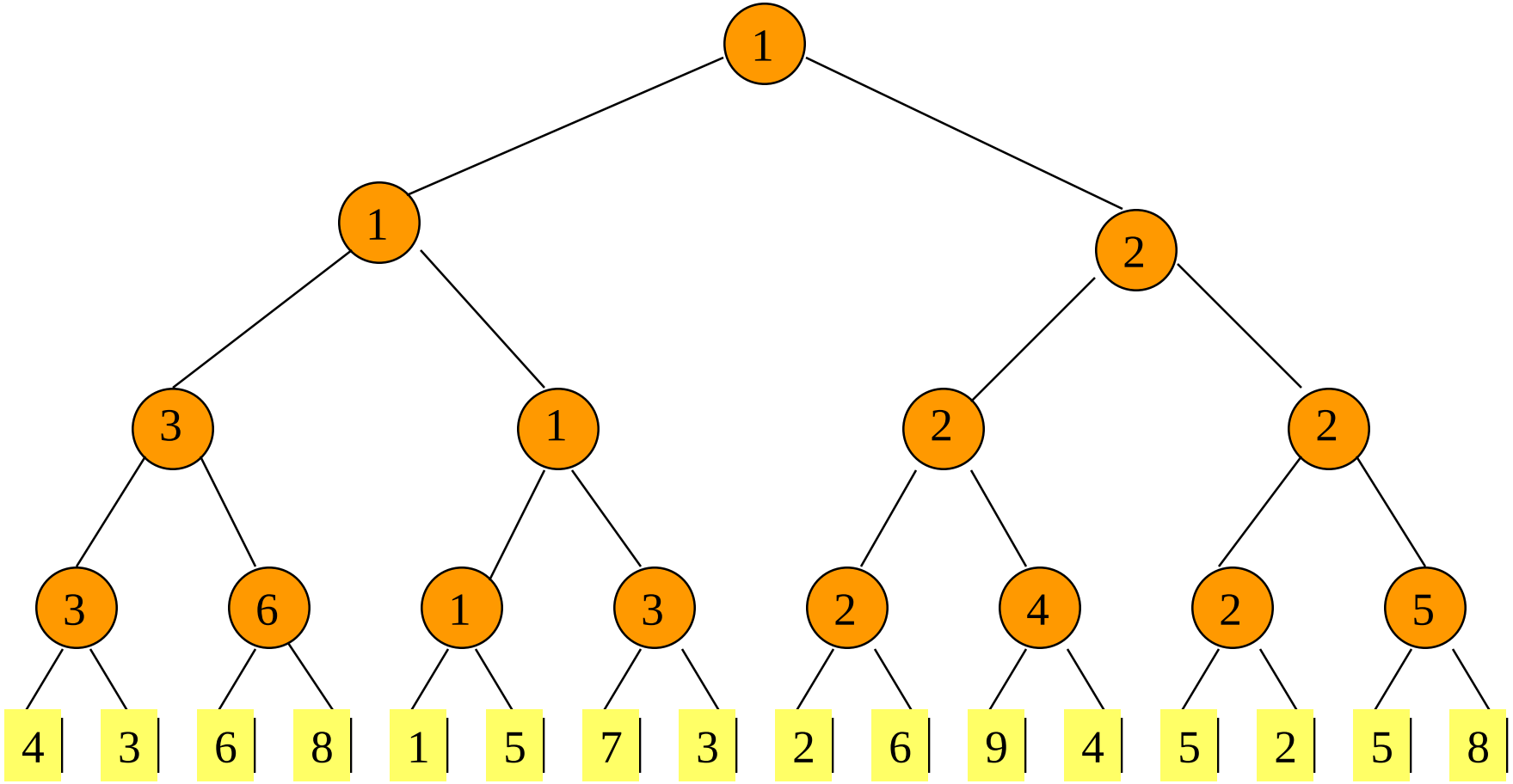
Each internal node represents a match played between its two children; the winner of the match is stored at the internal node.

Root has overall winner.

Winner Tree For 16 Players

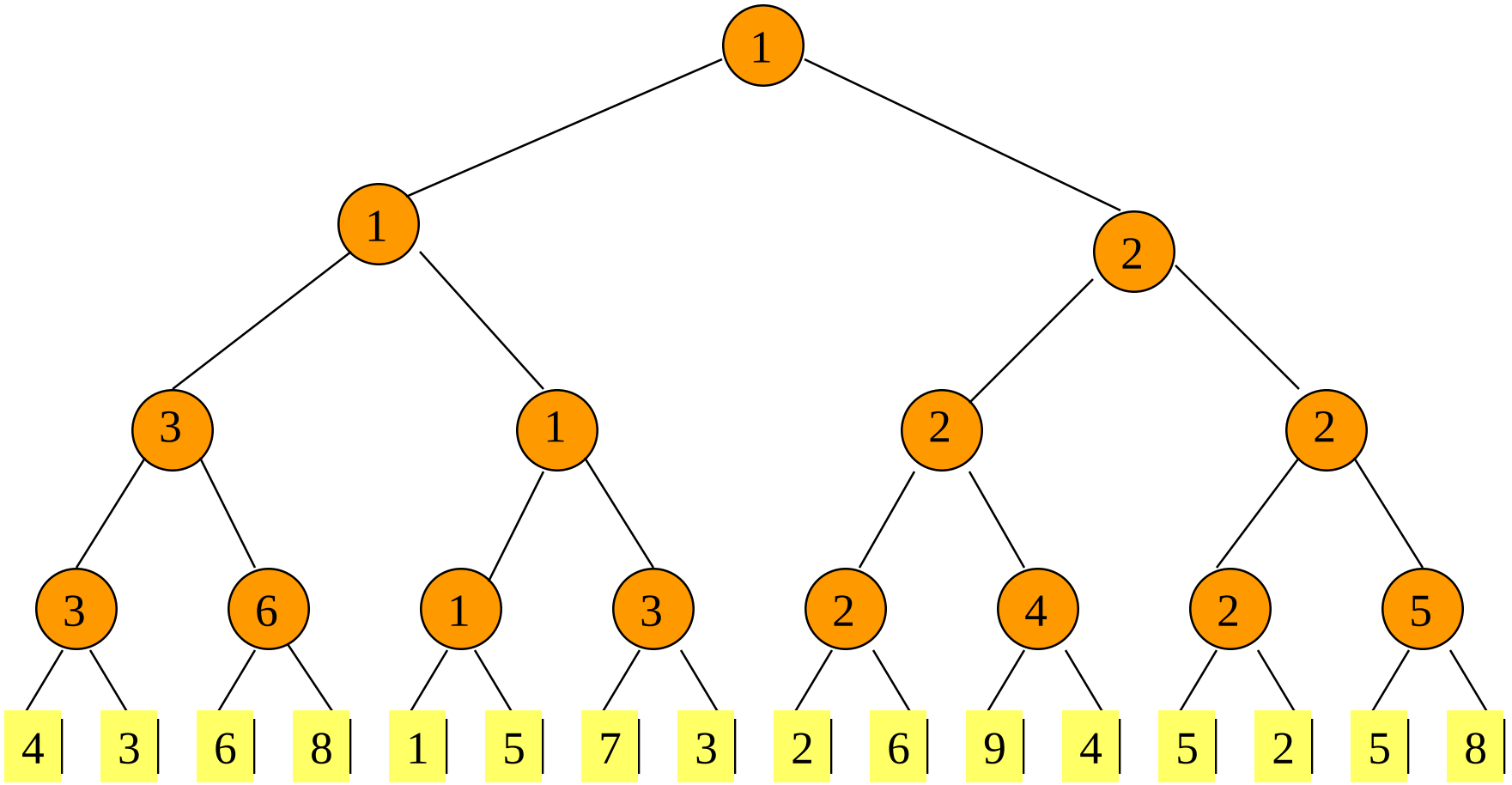


Winner Tree For 16 Players



Smaller element wins => min winner tree.

Winner Tree For 16 Players



height is $\log_2 n$ (excludes player level)

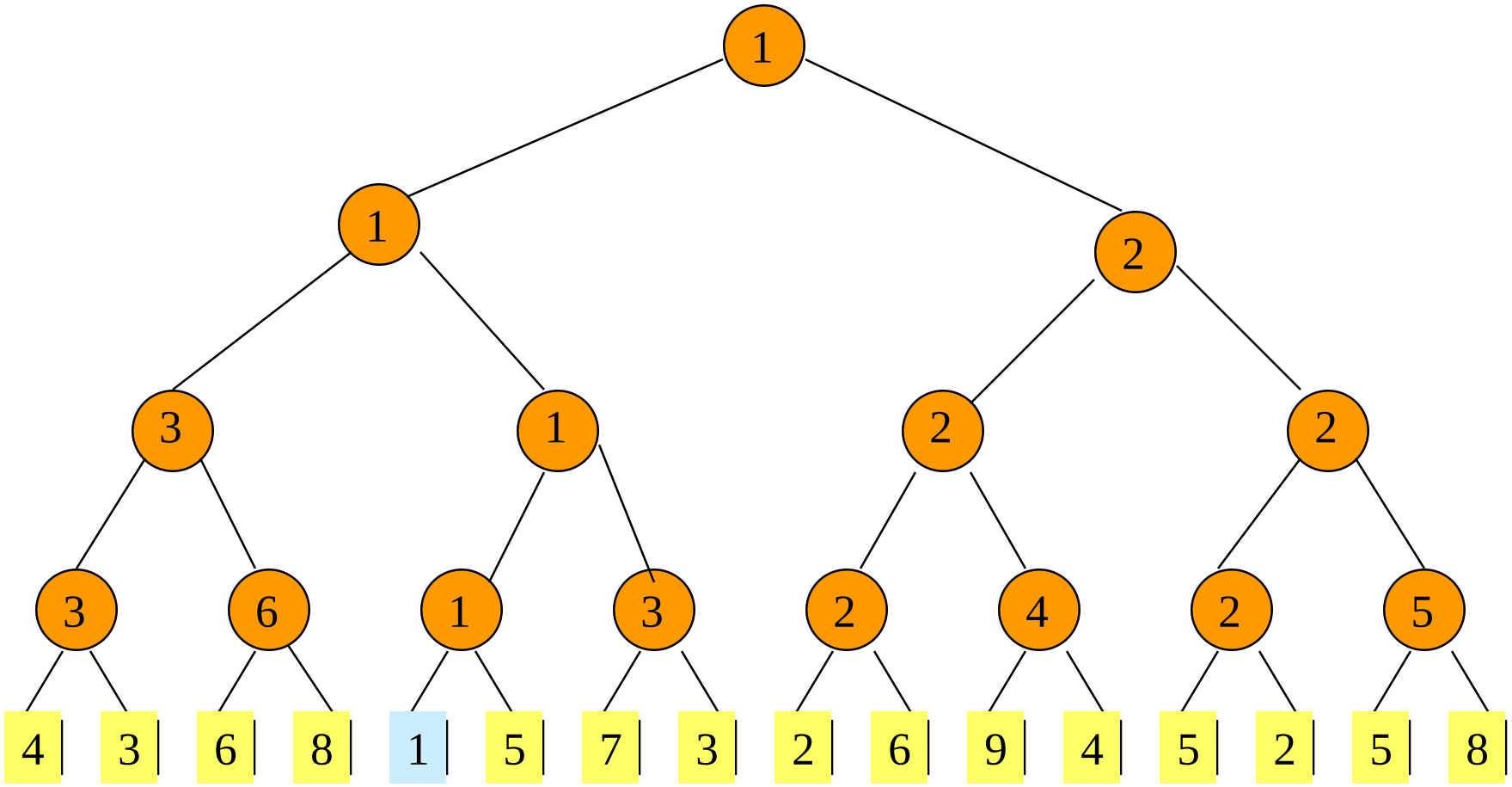
Complexity Of Initialize

- O(1) time to play match at each match node.
- n – 1 match nodes.
- O(n) time to initialize n-player winner tree.

Winner Tree Operations

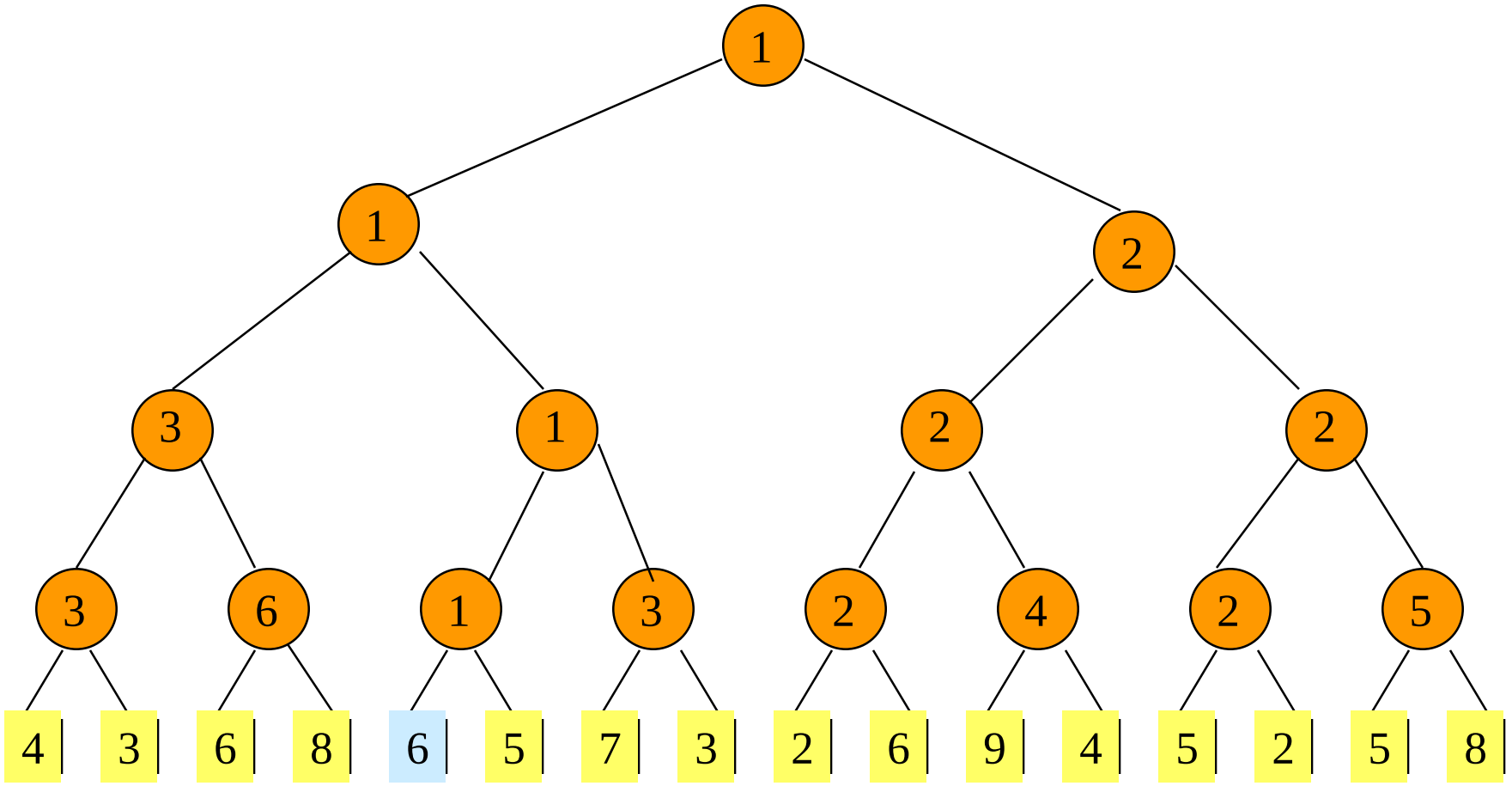
- Initialize
  - O(n) time
- Get winner
  - O(1) time
- Replace winner and replay
  - O(log n) time
  - More precisely Theta(log n)
- Tie breaker (player on left wins in case of a tie).

Replace Winner And Replay



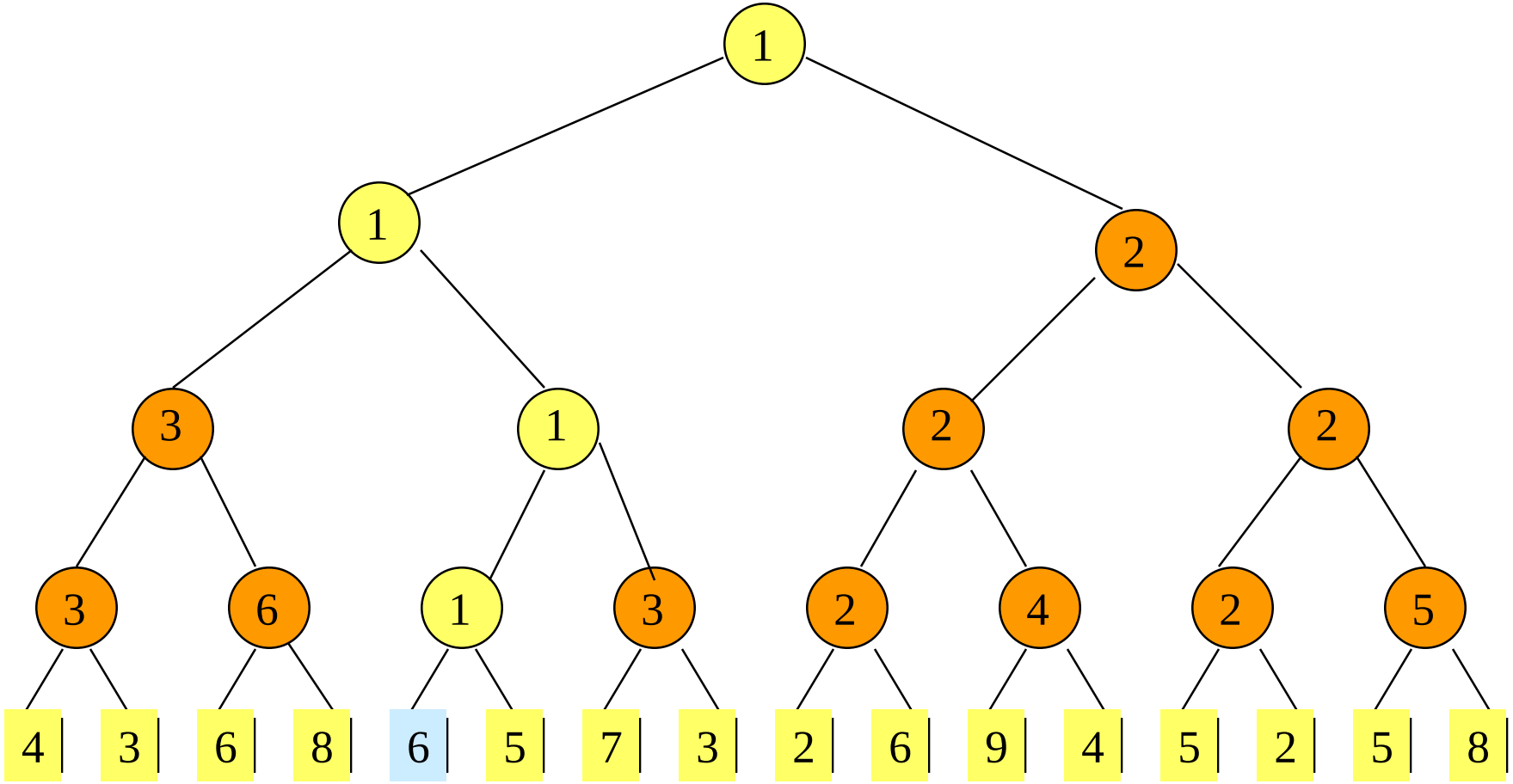
Replace winner with 6.

Replace Winner And Replay



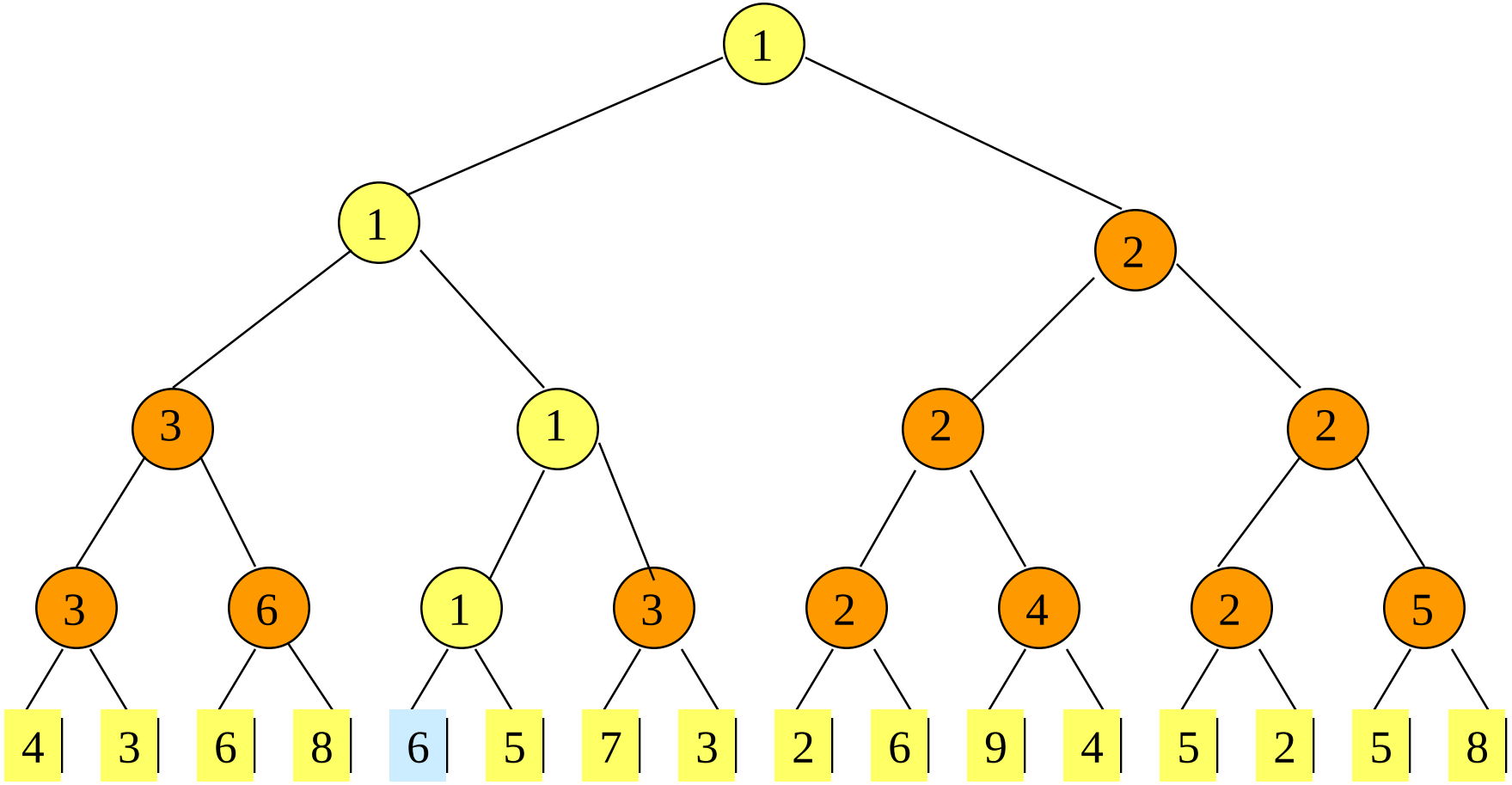
Replay matches on path to root.

Replace Winner And Replay



Replay matches on path to root.

Replace Winner And Replay

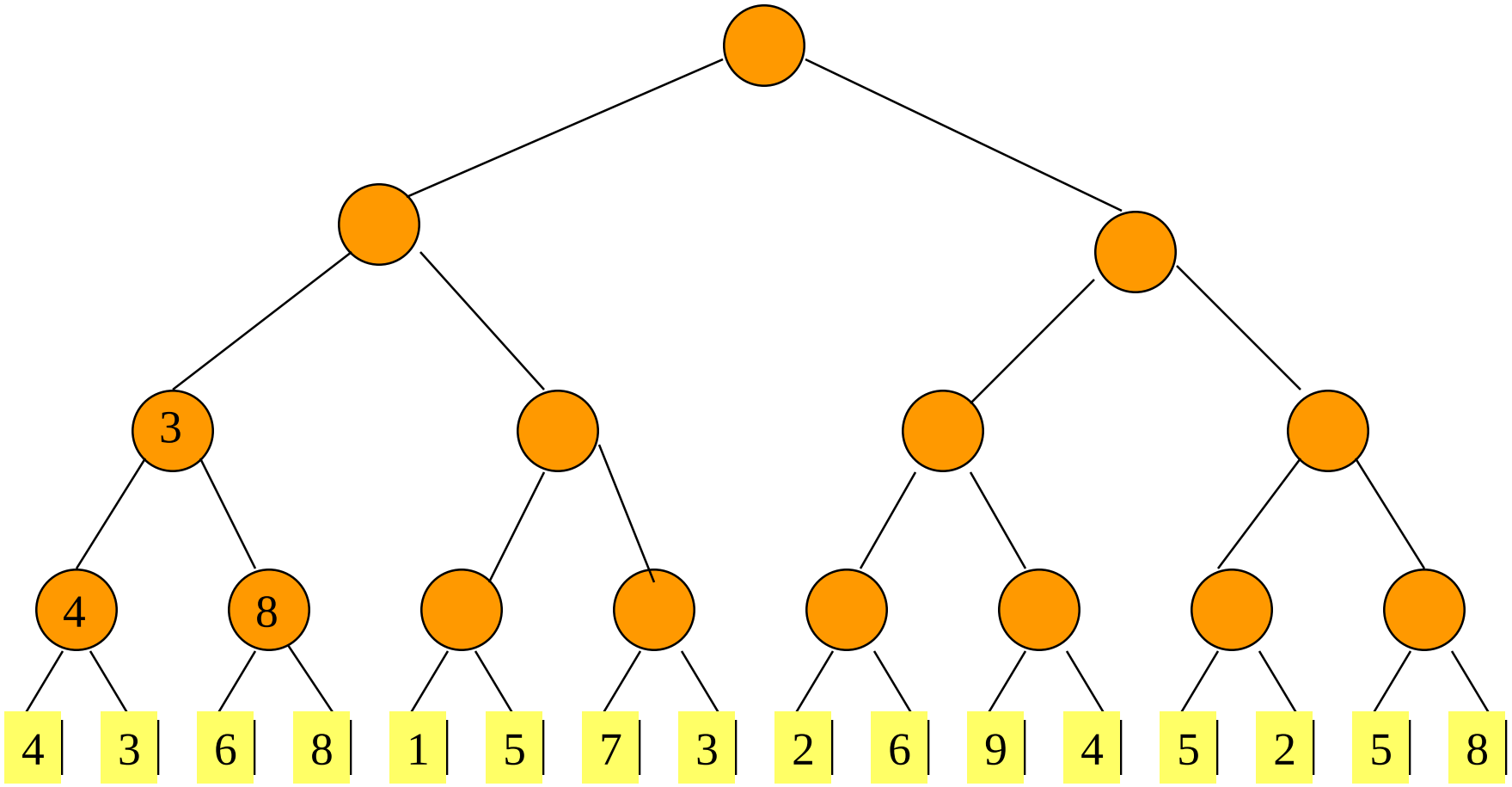


Opponent is player who lost last match played at this node.

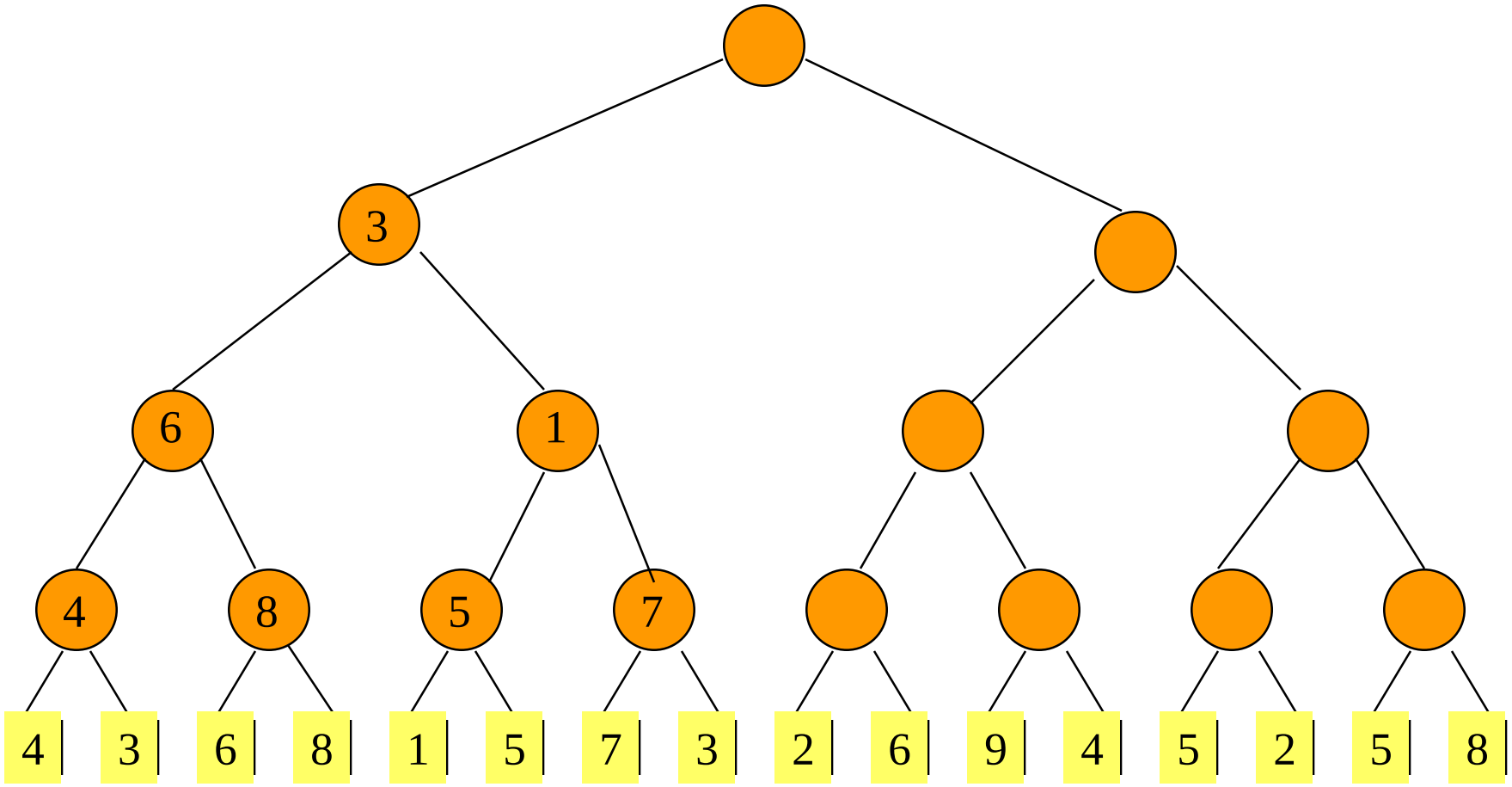
Loser Tree

Each match node stores the match loser rather than the match winner.

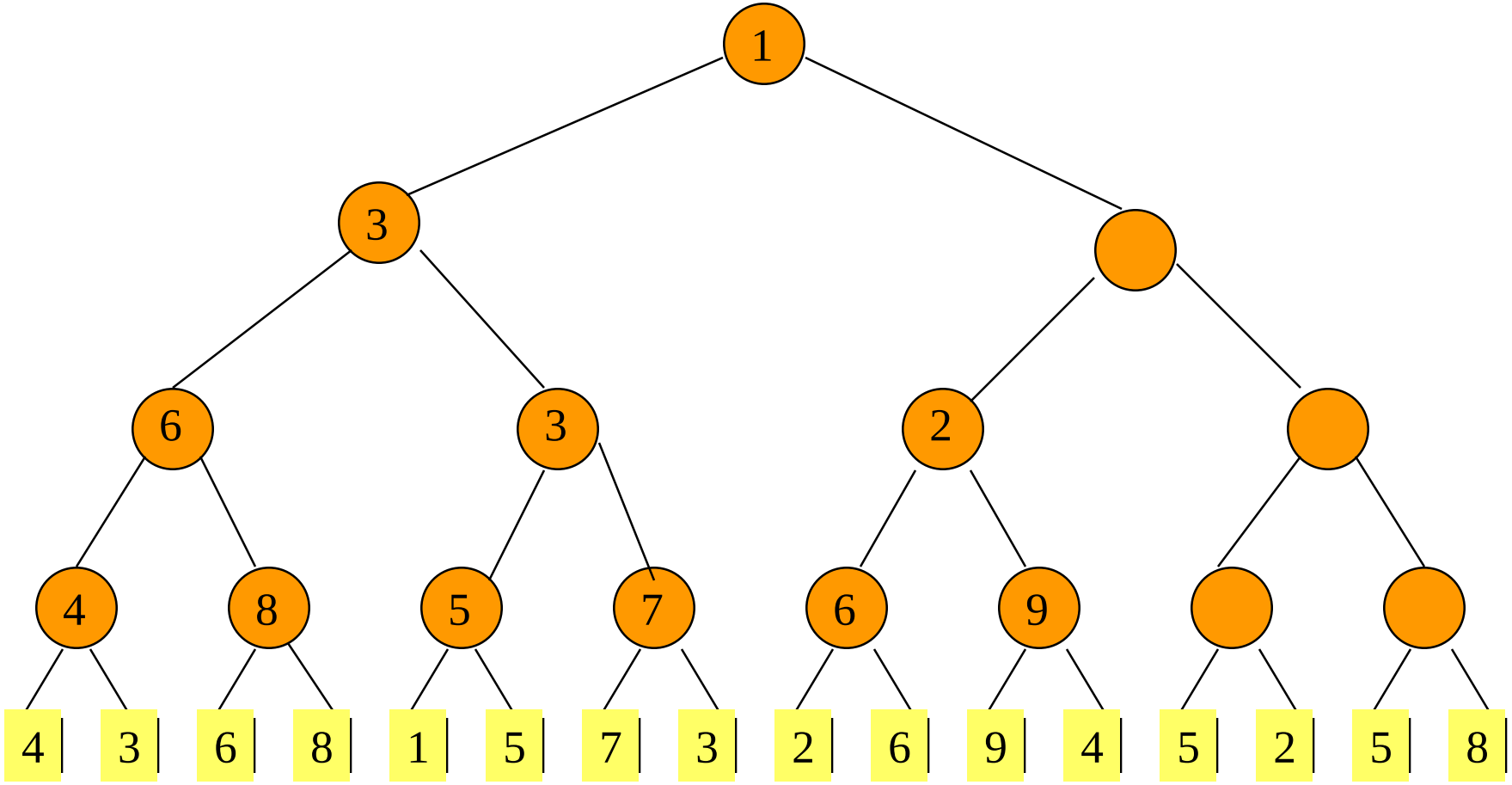
Min Loser Tree For 16 Players



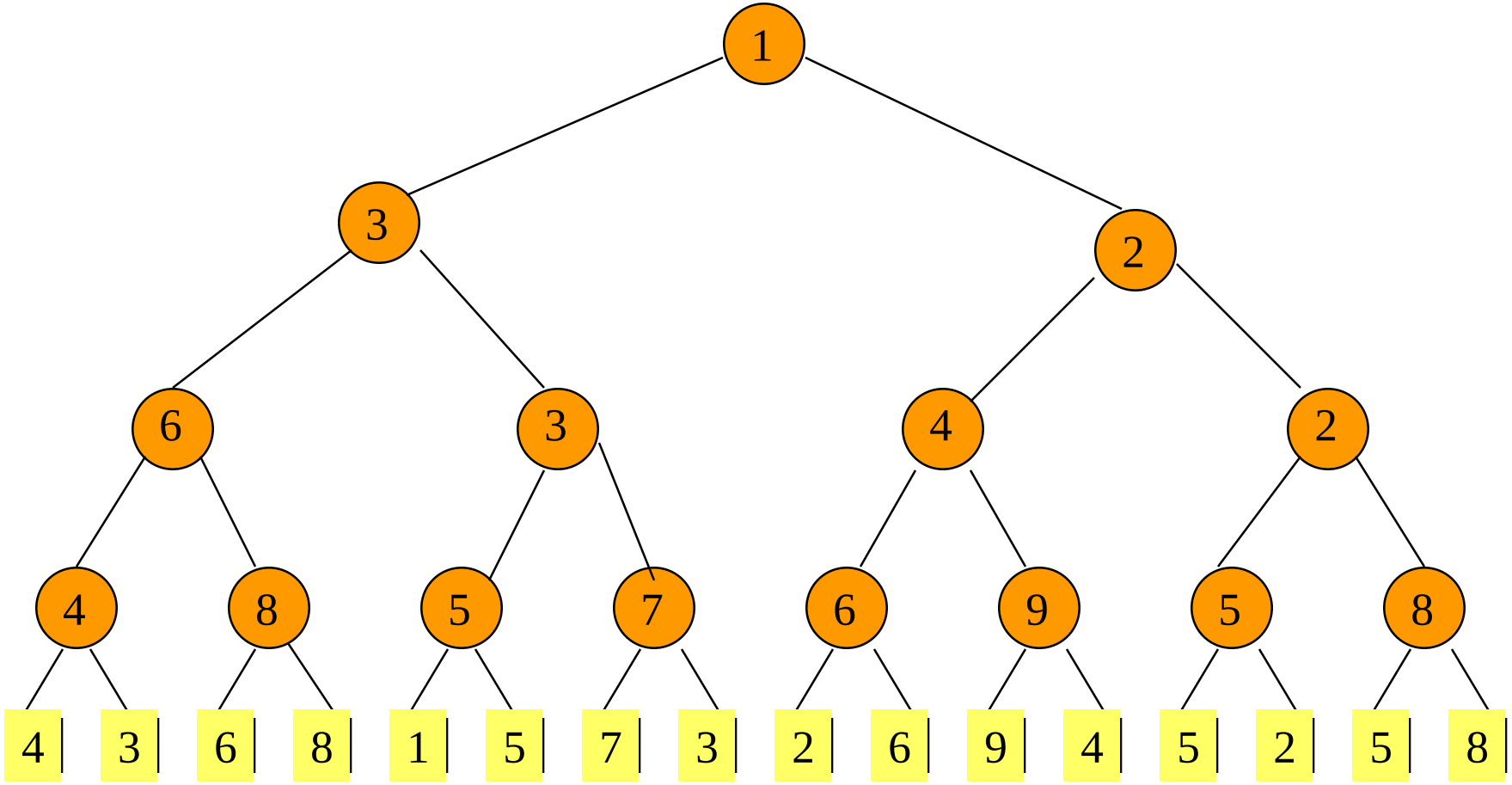
Min Loser Tree For 16 Players



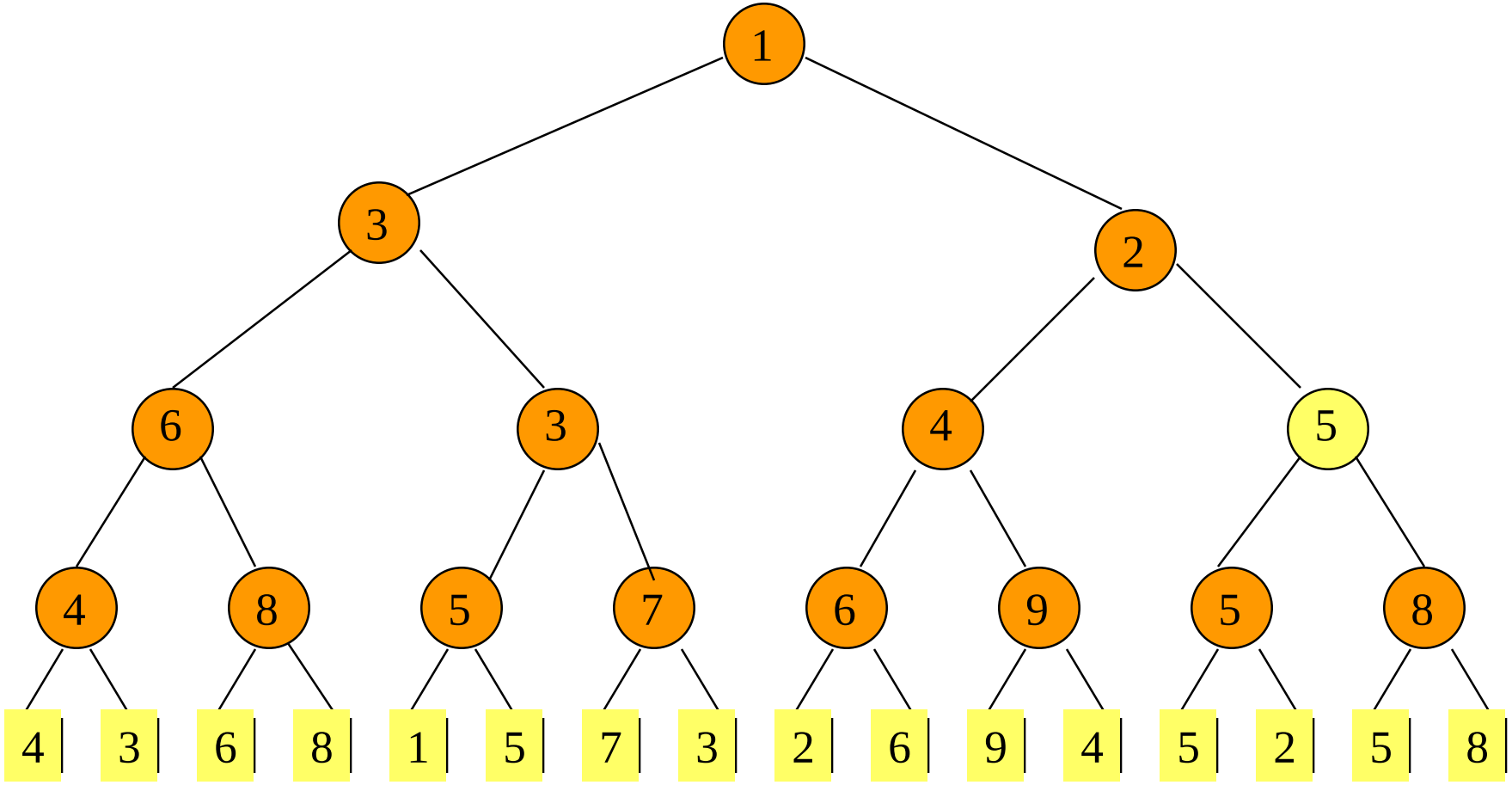
Min Loser Tree For 16 Players



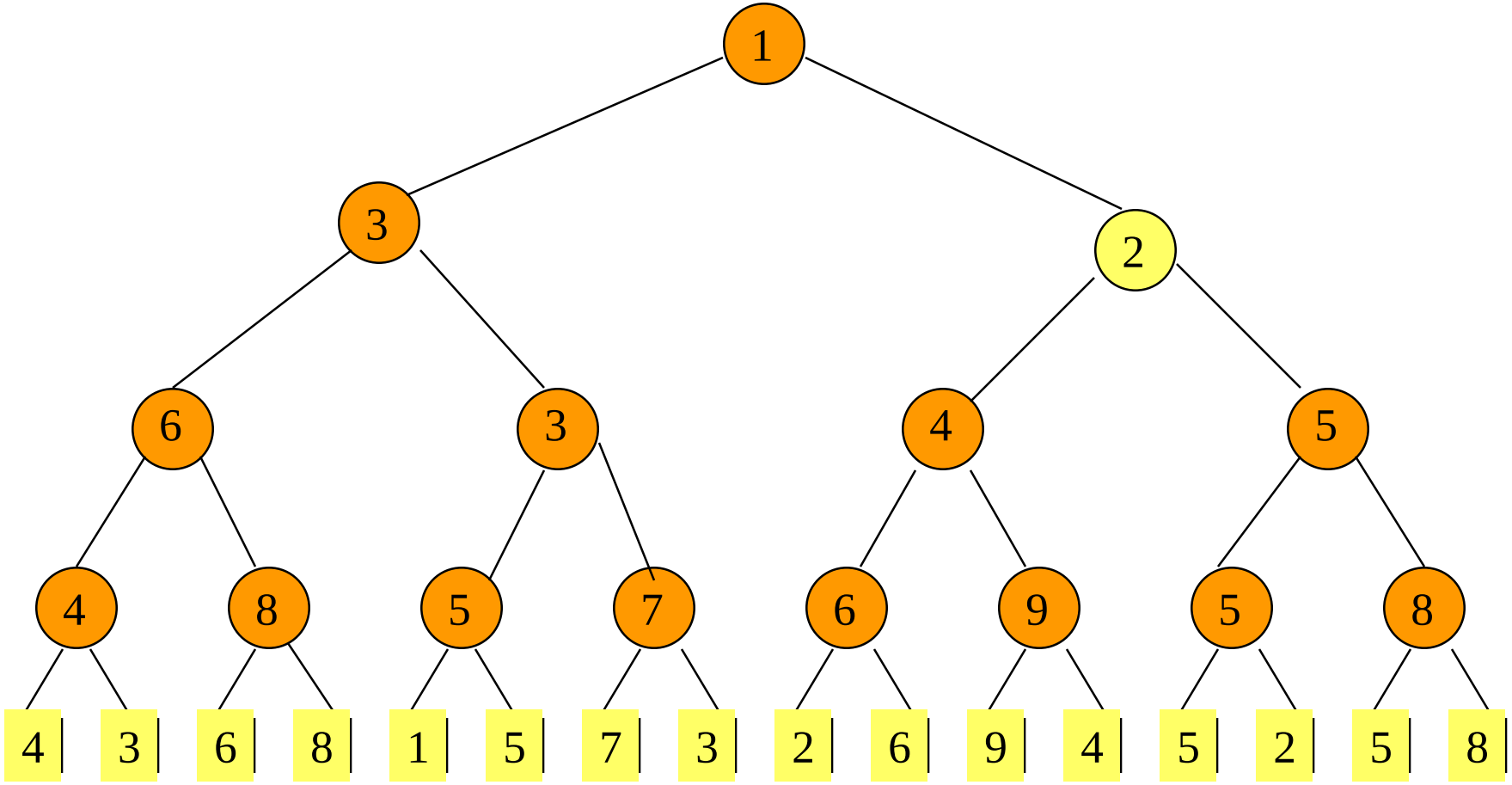
Min Loser Tree For 16 Players



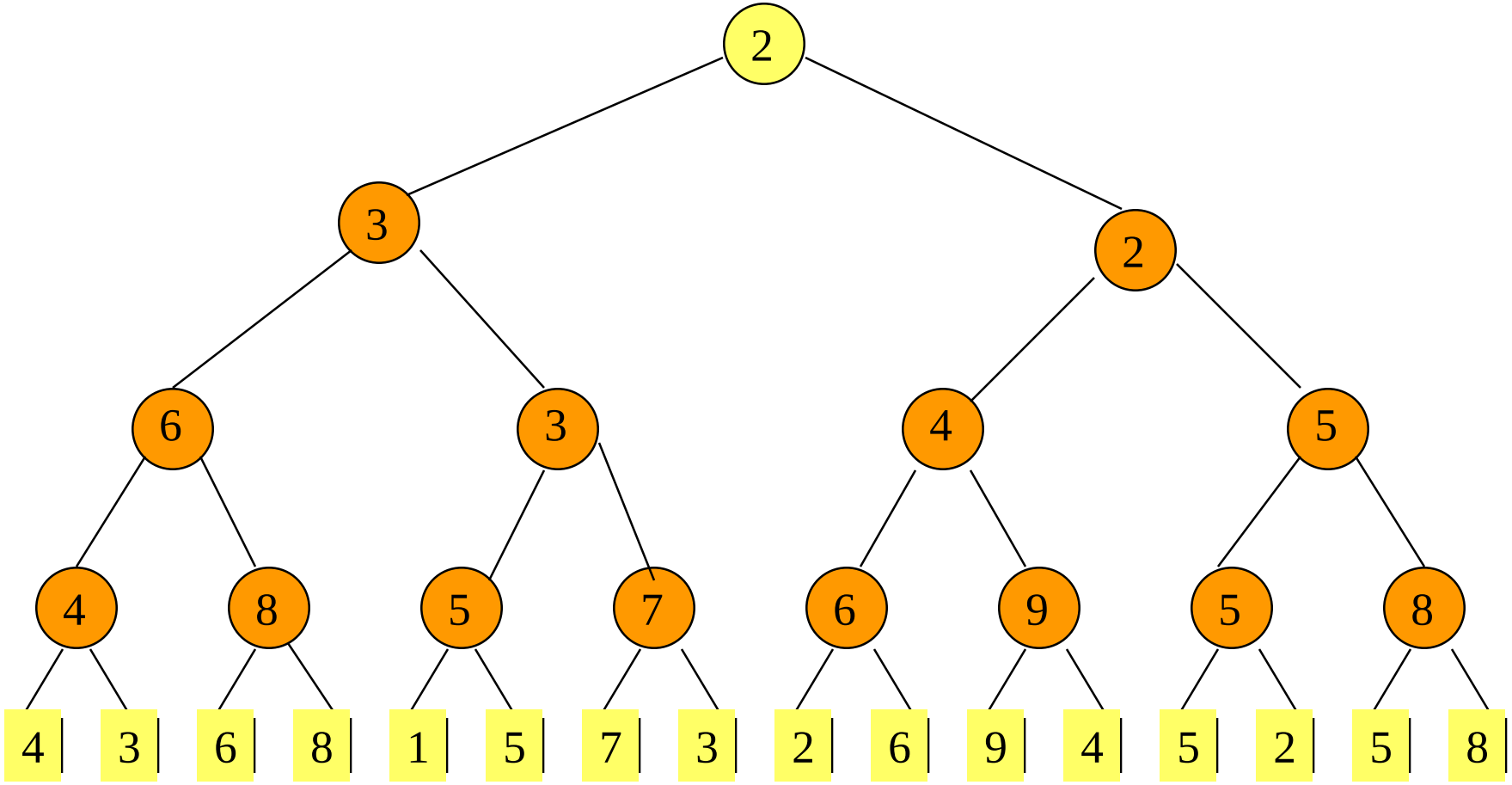
Min Loser Tree For 16 Players

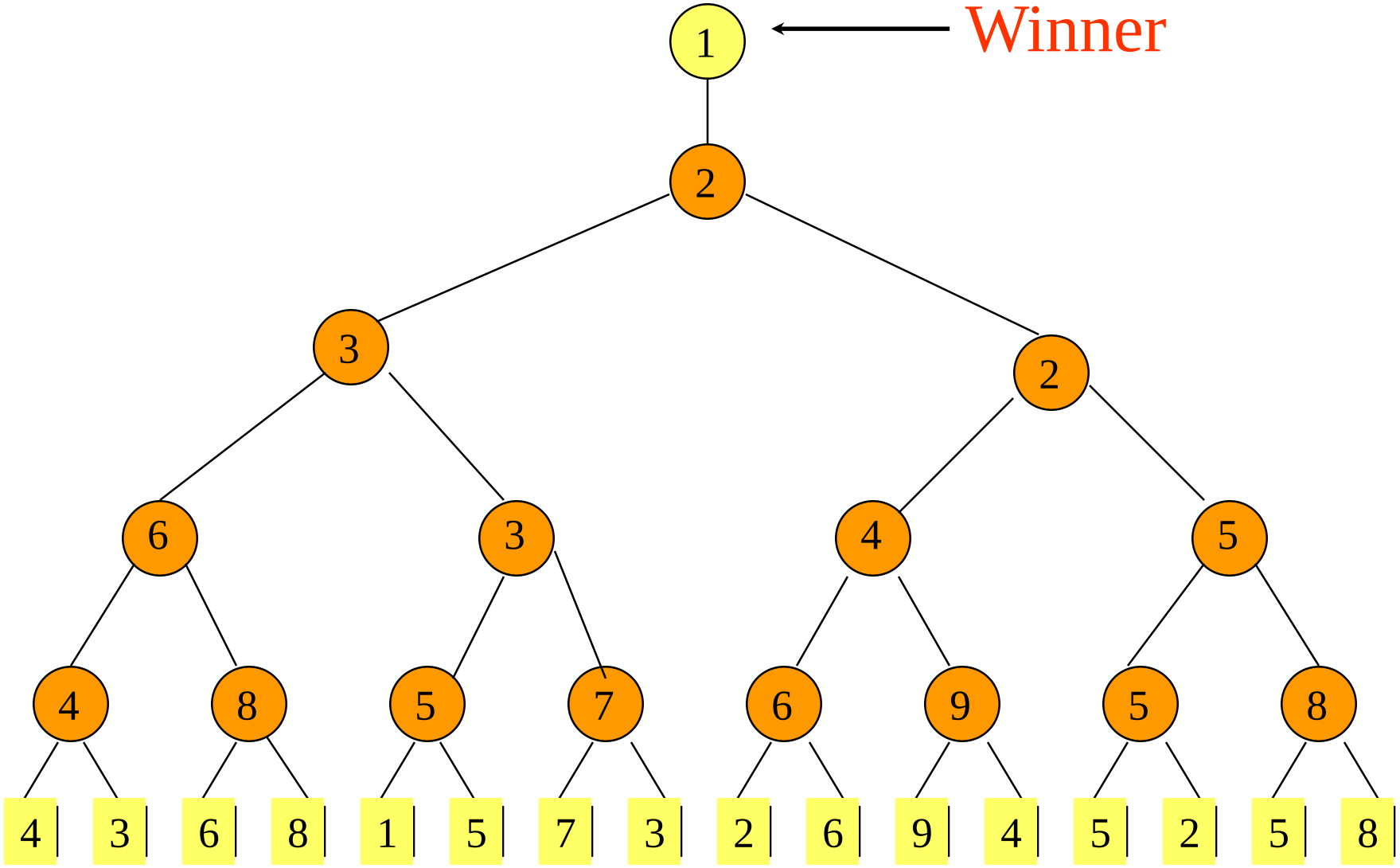


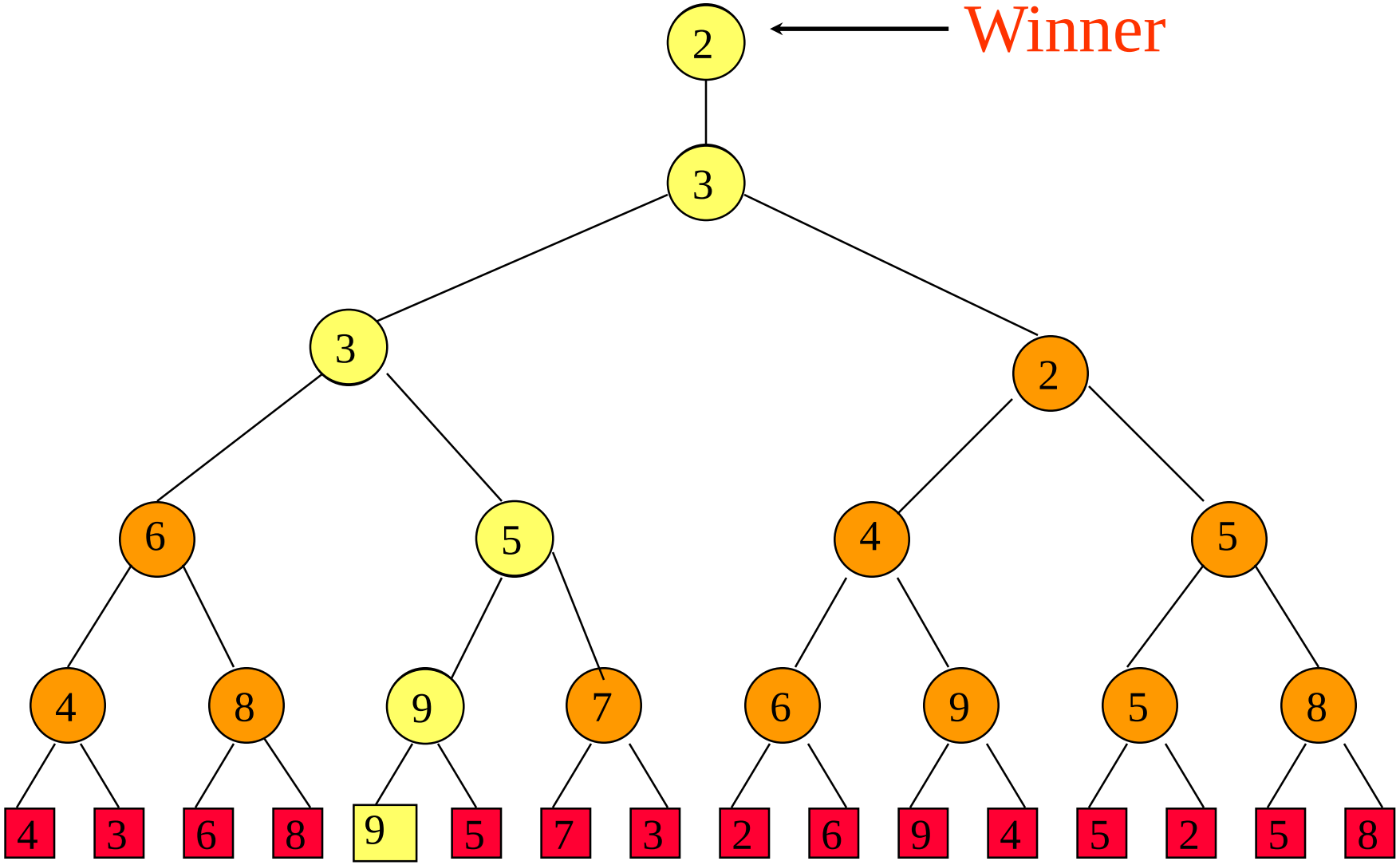
Min Loser Tree For 16 Players



Min Loser Tree For 16 Players




Winner
1
2
3
2
6
3
4
5
4
8
5
7
6
9
5
8
4
3
6
8
1
5
7
3
2
6
9
4
5
2
5
8



Replace winner with 9 and replay matches.

Complexity Of Replay

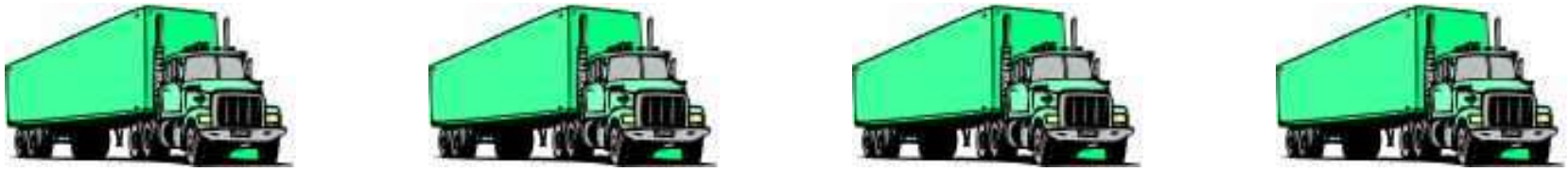


- One match at each level that has a match node.
- O(log n)
- More precisely Theta(log n).

Tournament Tree Applications

- Run generation.
- k-way merging of runs during an external merge sort.
- Truck loading.

Truck Loading





- n packages to be loaded into trucks
- each package has a weight
- each truck has a capacity of c tons
- minimize number of trucks

Bin Packing

- n items to be packed into bins
- each item has a size
- each bin has a capacity of c tons
- minimize number of bins

Bin Packing

Truck loading is same as bin packing.

- Truck is a bin that is to be packed (loaded).
- Package is an item/element.

Bin packing to minimize number of bins is NP-hard.

Several fast heuristics have been proposed.

Bin Packing Heuristics

- First Fit.
  - Bins are arranged in left to right order.
  - Items are packed one at a time in given order.
  - Current item is packed into leftmost bin into which it fits.
  - If there is no bin into which current item fits, start a new bin.

Bin Packing Heuristics

- First Fit Decreasing.
  - Items are sorted into decreasing order.
  - Then first fit is applied.

Bin Packing Heuristics

- Best Fit.
  - Items are packed one at a time in given order.
  - To determine the bin for an item, first determine set S of bins into which the item fits.
  - If S is empty, then start a new bin and put item into this new bin.
  - Otherwise, pack into bin of S that has least available capacity.

Bin Packing Heuristics

- Best Fit Decreasing.
  - Items are sorted into decreasing order.
  - Then best fit is applied.

Performance

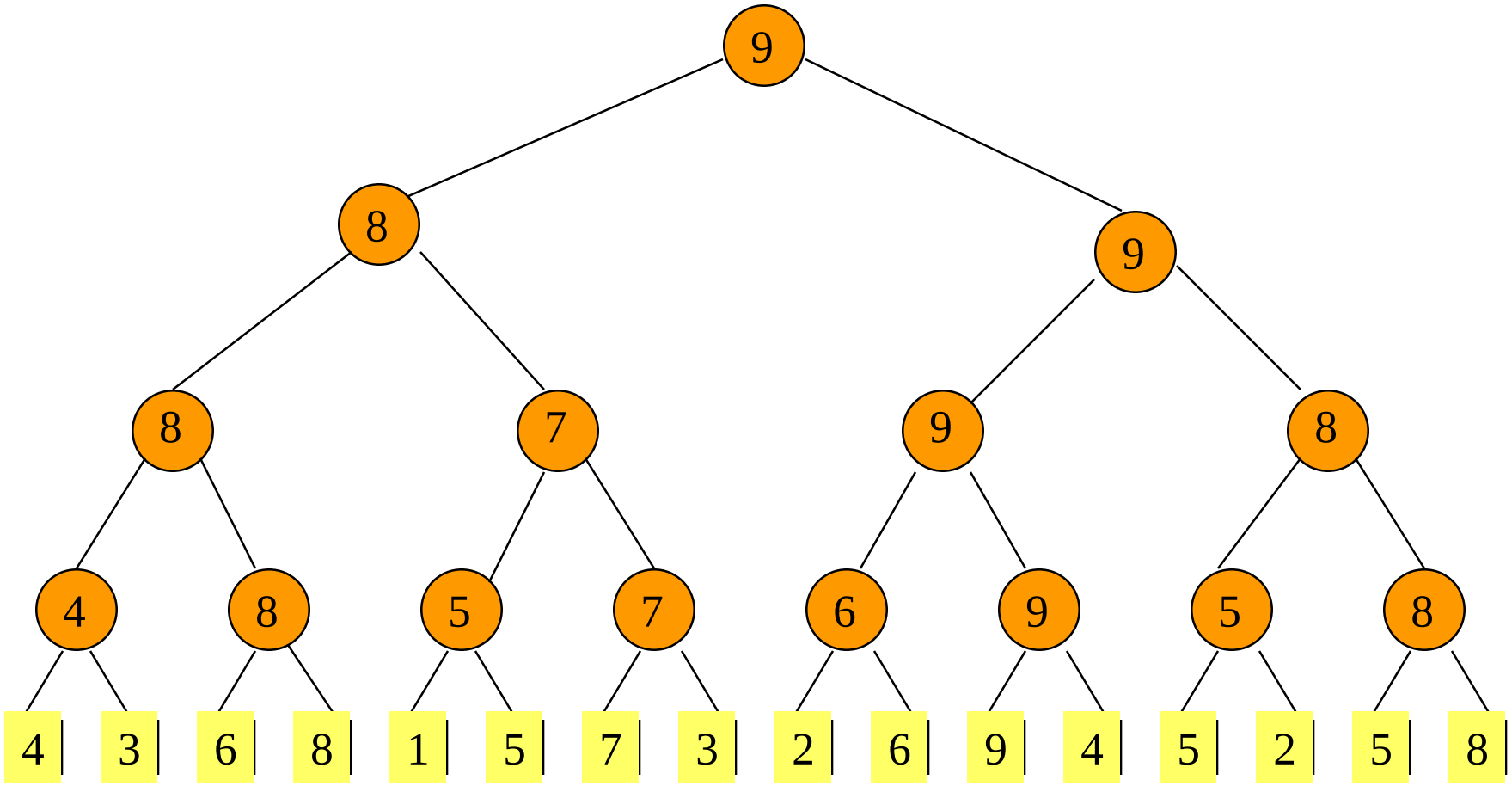
- For first fit and best fit:

  Heuristic Bins <= (17/10)(Minimum Bins) + 2

- For first fit decreasing and best fit decreasing:

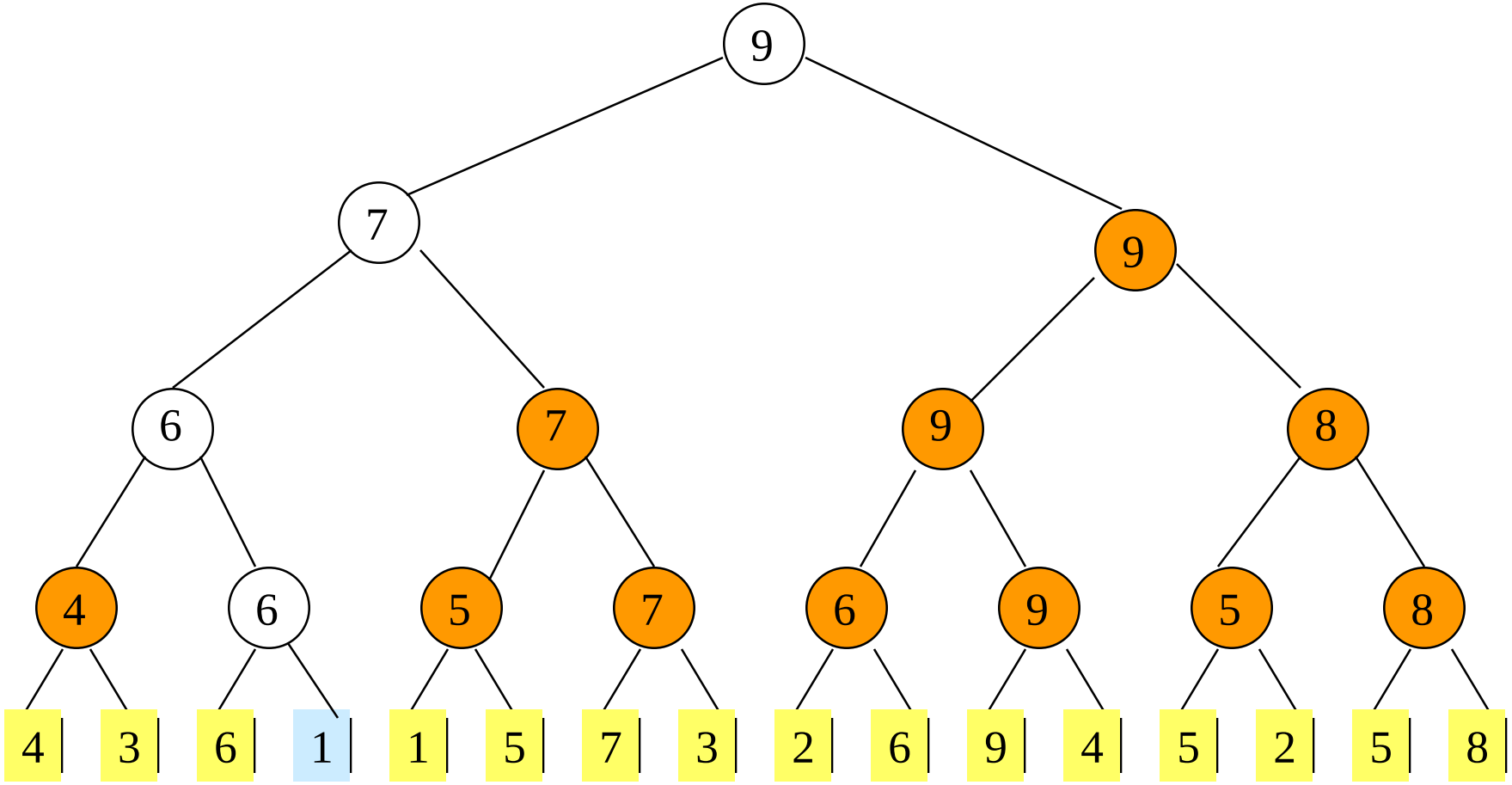
  Heuristic Bins <= (11/9)(Minimum Bins) + 4

Max Winner-Tree For 16 Bins



Item size = 7

Max Winner-Tree For 16 Bins



Acknowledgements

- E. Horowitz, S. Sahni, and D. Mehta, Fundamentals of data structures in C++, Second Edition, Silicon Press, 2007.
- Lecture Slides of Prof Sartaj Sahni.